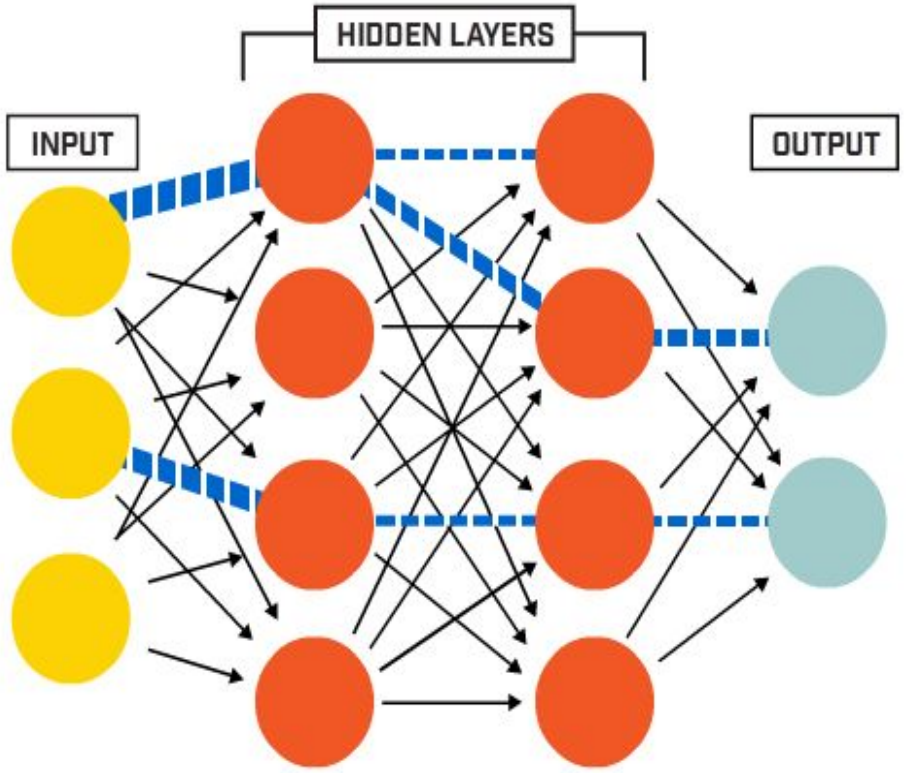
RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

UNIT - IV



Prof. Nithyashree G D
Dept of CSE
RVCE

Unit -4

- **Neural Networks**
  - What is NN
  - Models of neurons and network architecture
- **Single layer perceptron**
  - perceptron
- **Multilayer Perceptron**
  - Back propagation algorithm
  - Sequential and batch modes of training, stopping criteria.

What is a Neural Network?

- Inspired by the human brain
- Consists of interconnected nodes (neurons)
- Learns from data to make predictions/decisions
- Applications: Image recognition, NLP, robotics, etc.

Unit -4

Biological Neuron vs. Artificial Neuron

Biological Neuron

- Dendrites (inputs)
- Soma (processing)
- Axon (output)

Artificial Neuron

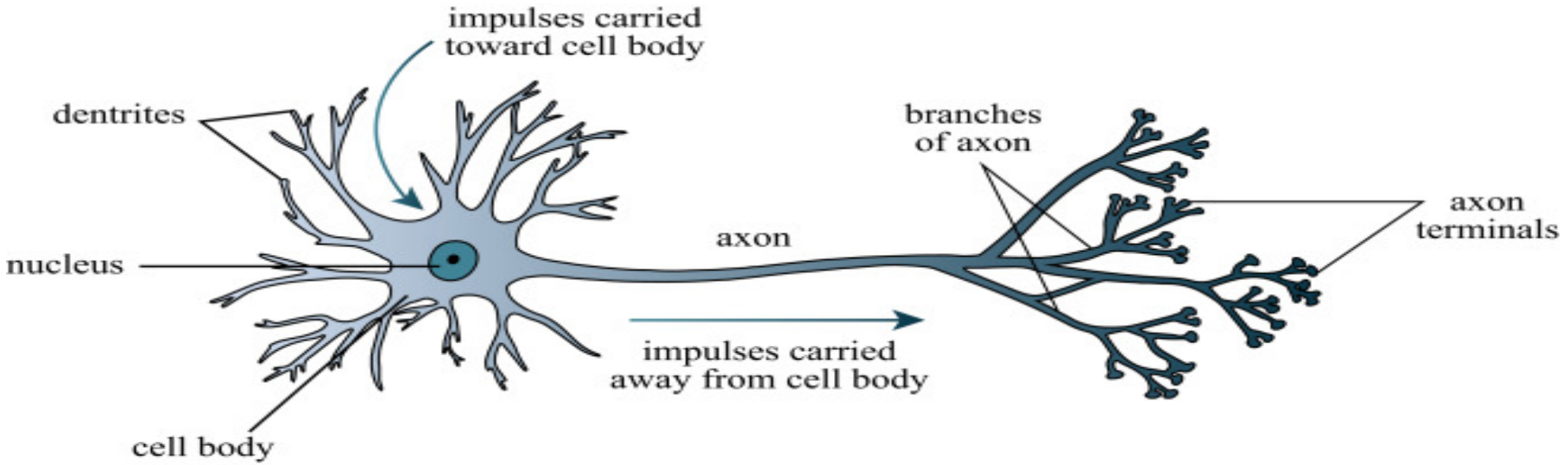
- Inputs (x1, x2, …, xn)
- Weights (w1, w2, …, wn)
- Summation + Activation Function
- Output (y)

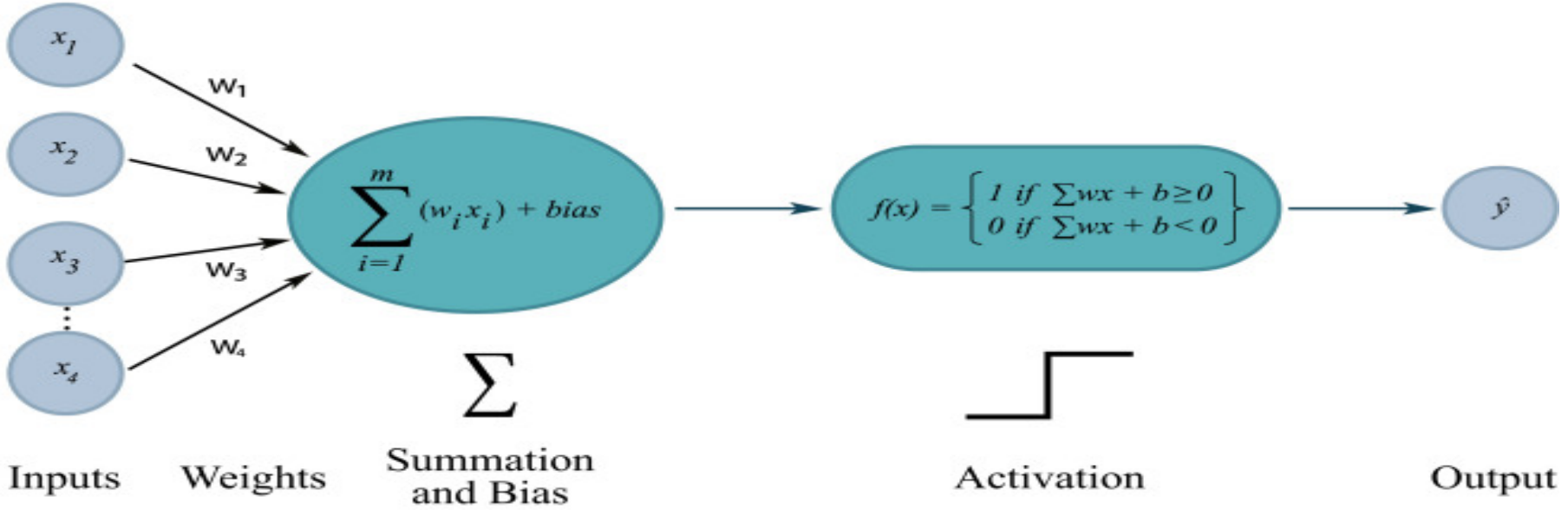
Mathematical Model of a Neuron

- Input: $x_1, x_2, \ldots, x_n$
- Weights: $w_1, w_2, \ldots, w_n$
- Net Input: $z = \sum w_i x_i + b$
- Activation Function: f(z)
- Output: $y = f(z)$

Activation Functions: Step, Sigmoid, ReLU, Tanh

Unit -4


impulses carried
toward cell body
dentrites
branches
of axon
axon
terminals
nucleus
axon
impulses carried
away from cell body
cell body


x_1
x_2
x_3
x_4
w_1
w_2
w_3
w_4
$\sum_{i=1}^{m}(w_i x_i) + bias$
$f(x) = \begin{cases} 1 \text{ if } \sum wx + b \geq 0 \\ 0 \text{ if } \sum wx + b < 0 \end{cases}$
$\hat{y}$
Inputs
Weights
Summation
and Bias
Activation
Output

Unit -4

- **Biological Neuron**
- **Dendrites** → Receive input signals from other neurons.
- **Cell body (soma)** → Processes signals.
- **Nucleus** → Controls the neuron.
- **Axon** → Carries the processed signal away.
- **Axon terminals** → Pass the signal to the next neuron.
- So: **Input → Processing → Output.**

Artificial Neuron: In ANN, we mimic the above structure with math:

•**Inputs (x_1, x_2, x_3, x_4):** → They are the data/features you feed (e.g., height, weight, age).

•**Weights (w_1, w_2, w_3, w_4)**
•→ Strength of each connection (like synaptic strength in the brain).

Summation + Bias ($\Sigma w_i x_i + bias$)
•→ Like the cell body combining signals.
→ Bias shifts the decision boundary (like a neuron's internal threshold).

•**Activation Function f(x):** → Decides whether the neuron "fires" or not.
→It's a **step function** (1 or 0).
→ In modern ANN, we often use **ReLU, sigmoid, tanh** instead.

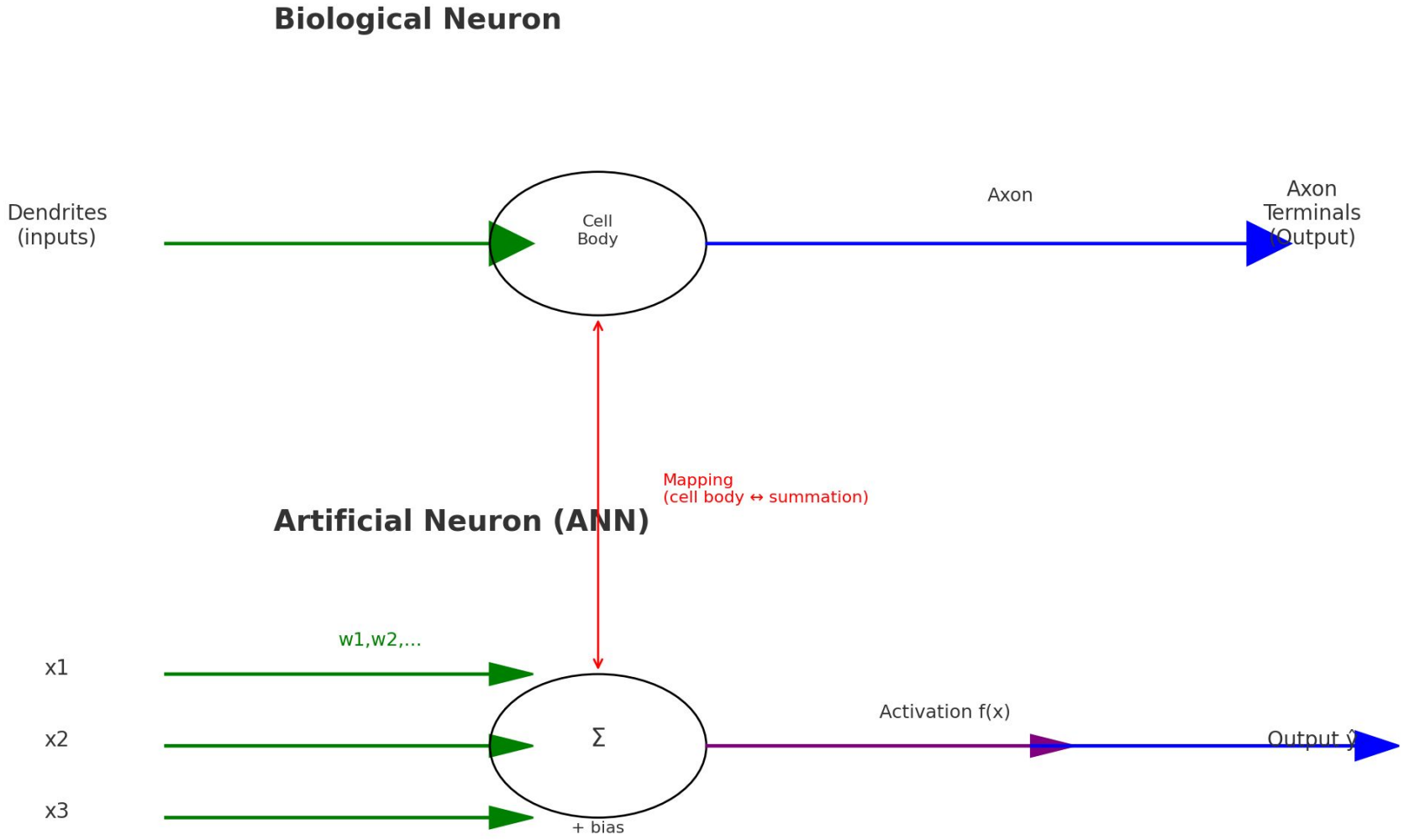
•**Output ($\hat{y}$)**
•→ Signal sent forward (like axon sending impulses).
→ Becomes input for the next neuron in the network.

Unit -4

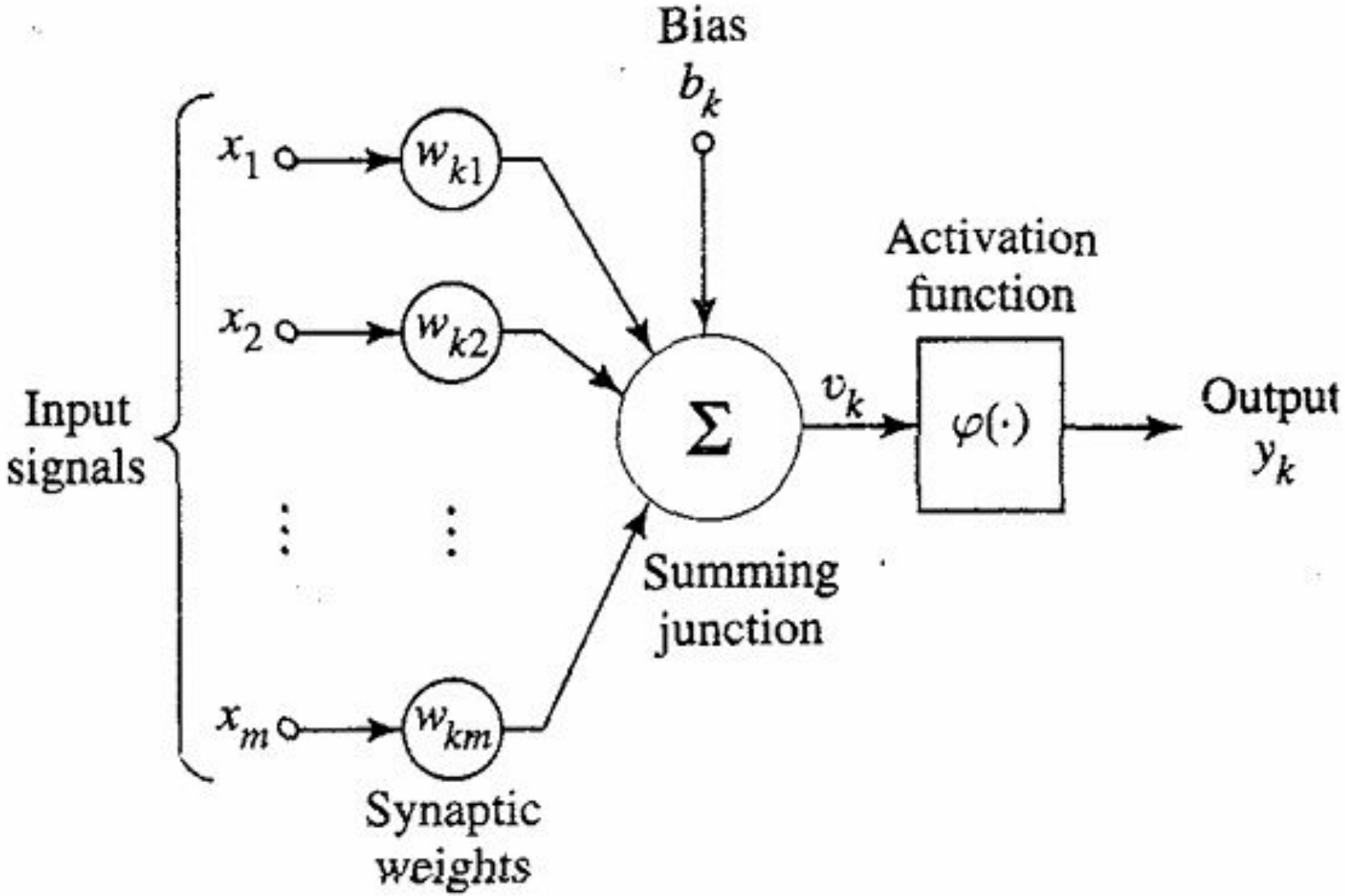
Mapping Between Biological and Artificial Neuron

Biological Neuron	Artificial Neuron (ANN)
Dendrites	Inputs ($x_1, x_2, \ldots, x_n$)
Synaptic Strengths	Weights ($w_1, w_2, \ldots, w_n$)
Cell Body (Soma)	Summation & Bias ($\Sigma w_i x_i + b$)
Firing Rule	Activation Function f(x)
Axon	Output ($\hat{y}$)
Axon Terminals	Input to next neurons

Mapping Between Biological and Artificial Neuron

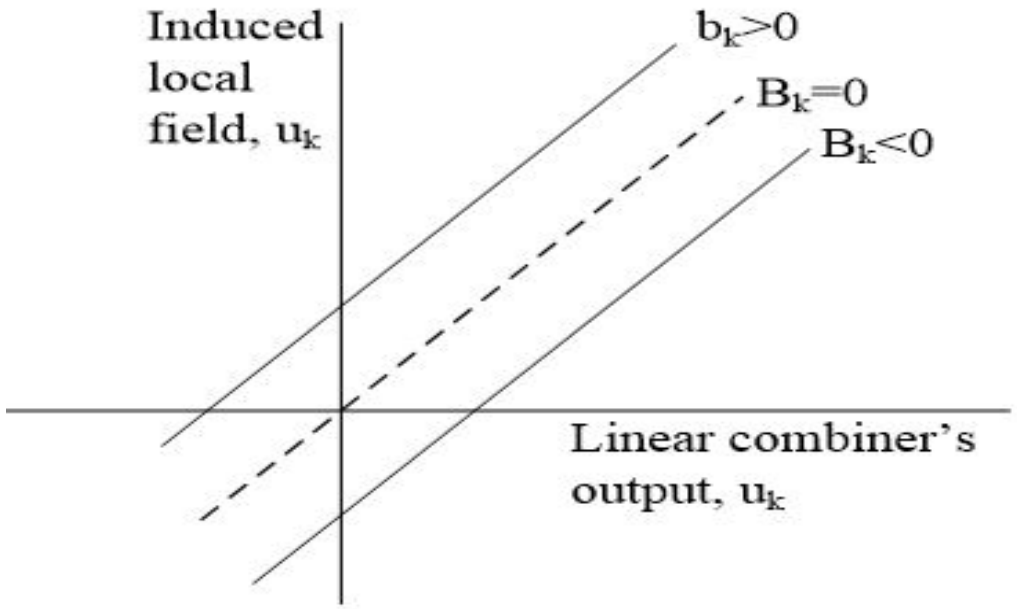


Unit -4

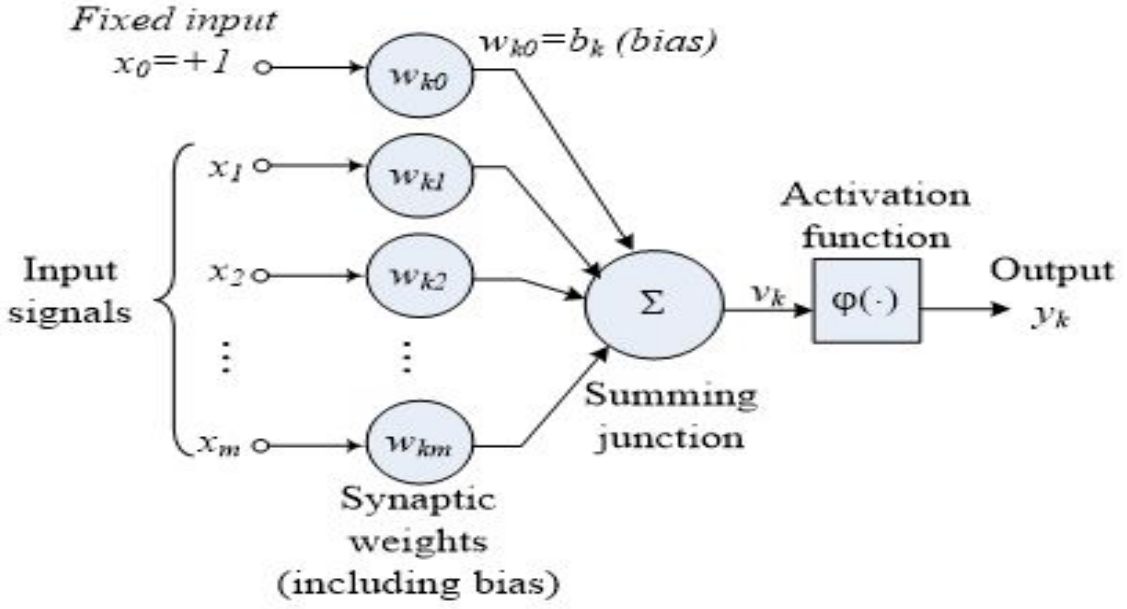

Bias
b_k
x_1
w_{k1}
x_2
w_{k2}
x_m
w_{km}
Input signals
Activation function
v_k
Σ
$\varphi(\cdot)$
Output
y_k
Summing junction
Synaptic weights

Unit -4

Non Linear Model of a Neuron



Affine transformation produced by the presence of a bias



Another Nonlinear model of a neuron

Unit -4

Functional Parameters of a Neuron

Function parameter	Description
w	Vector of real-valued weights on the connections
w · **x**	Dot product ($\sum_{i=1}^{n} w_i x_i$)
n	Number of inputs to the perceptron
b	The bias term (input value does not affect its value; shifts decision boundary away from origin)

Unit -4

Activation Function denoted by $\varphi(v)$ defines the output of a neuron in terms of induced local field v.

Threshold Function

$$\varphi(v) = \begin{cases} 1 & if\ v \geq 0 \\ 0 & if\ v < 0 \end{cases}$$

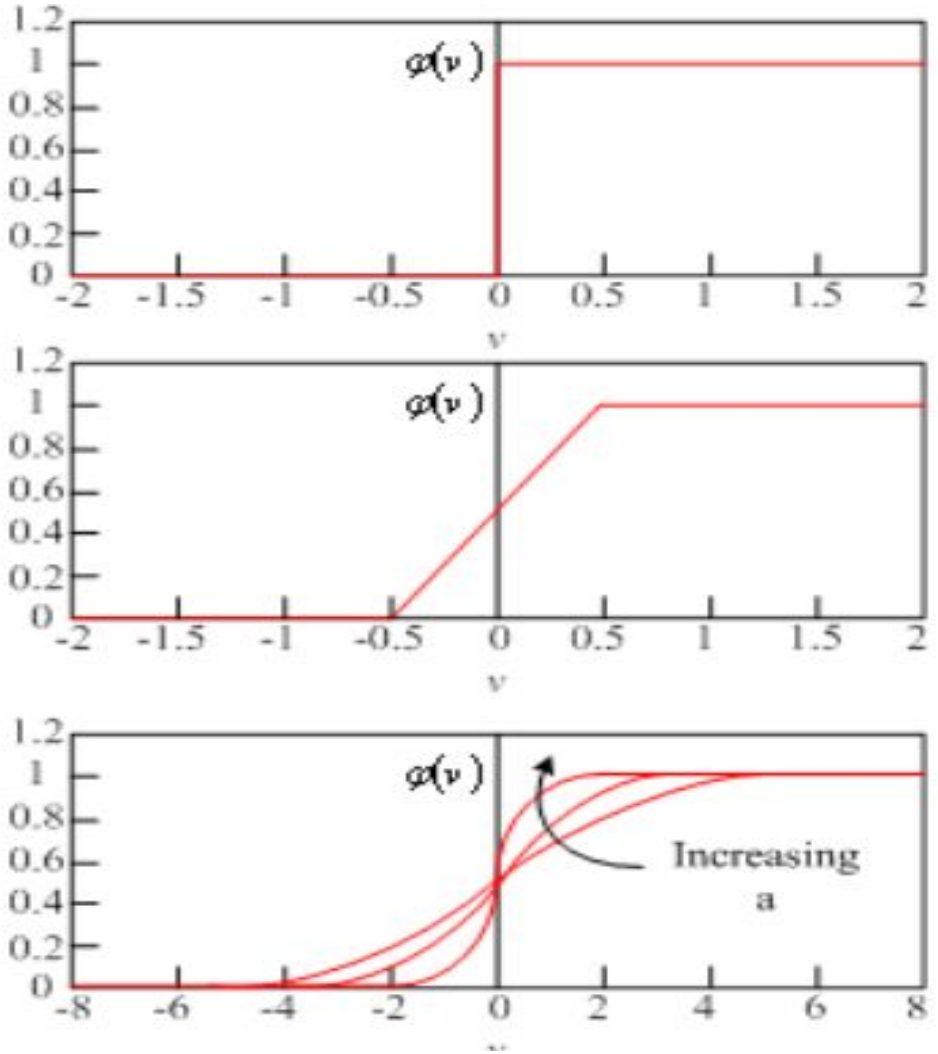
Piecewise-Linear Function

$$\varphi(v) = \begin{cases} 1 & v \geq 1/2 \\ v & -1/2 < v < 1/2 \\ 0 & v \leq -1/2 \end{cases}$$

Sigmoid Function

$$\varphi(v) = \frac{1}{1+\exp(-av)}$$

a is the slope parameter



Unit -4

- **Tanh (Hyperbolic Tangent):**

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- **Range:** (–1, 1)
- **Pros:** Zero-centered (better than sigmoid for optimization).
- **Cons:** Still suffers from vanishing gradients for large |x|.

- **ReLU (Rectified Linear Unit)**

$$f(x) = \max(0, x)$$

- **Range:** [0, ∞)
- **Pros:** Simple, computationally efficient, mitigates vanishing gradients, works well in deep nets.
- **Cons:** *Dying ReLU* problem (neurons stuck at 0 if weights push them negative).

Unit -4

- **Softmax**
- Used for **multi-class classification** in the output layer.
- Converts logits into probabilities:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Which Activation to Use?

- **Hidden layers (general):** ReLU or its variants (Leaky ReLU, GELU).
- **Output layer:**
  - **Binary classification:** Sigmoid
  - **Multi-class classification:** Softmax
  - **Regression:** Linear (no activation)

Unit -4

Neural Networks as Directed Graphs – Signal Flow Diagrams

- **Neural Network (NN)** can be represented as a **directed graph.**
- Each node and edge in this graph represents a mathematical or functional relationship between signals.

- Block Diagram Representation
- The **block diagram** provides a *functional overview* of the neural network.

 Each **block** represents a function or a neuron.

 The **arrows** indicate the **direction of data flow**.

 It provides a *high-level view* of the computation process.

Unit -4

Signal Flow Graph Representation

- The **signal flow graph (SFG)** gives a *complete mathematical and directional* representation of the neural network.
- It shows **nodes** and **directed edges (arrows)** between them.
- Each edge represents a *signal transmission* or *weight connection*.
- The arrows define the **direction of signal propagation** — typically from input to output.

- **(x1) → (Neuron 1) → (y1)**
- **(x2) → (Neuron 2) → (y2)**

Unit -4

Architectural Graph Representation

This diagram illustrates the layout or topology of the network, showing how the layers and neurons are interconnected.

Input Nodes → Hidden Nodes → Output Node

Explanation:

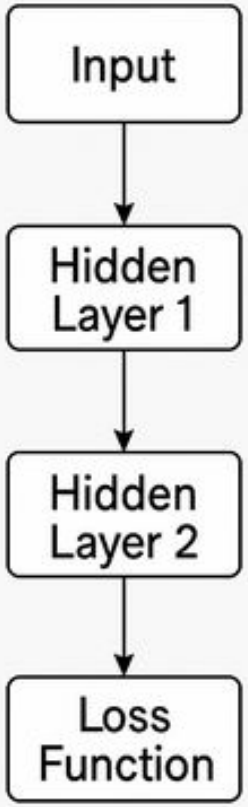
Input layer: Nodes that take input signals (x_1, x_2, …).

Hidden layer(s): Nodes that process signals via activation functions.

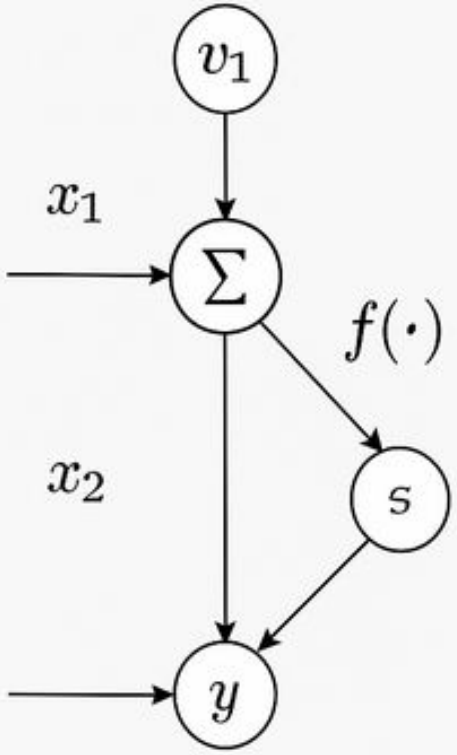
Output layer: Produces the final result or decision.

Unit -4

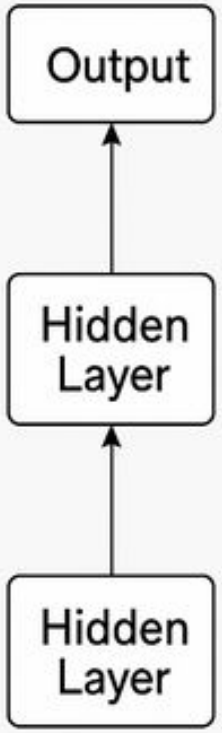
Block Diagram



Signal Flow Graph



Architectural Graph



Unit -4

Description of the Graph

Source Nodes:

These supply **input signals** to the network (e.g., features like x_1, x_2). They represent where data enters the system.

Computation Nodes:

Each **neuron** is represented as a single node that performs computation.

These are sometimes called **processing elements**.

Each node performs:

$$y_j = f\left(\sum_i w_{ji} x_i + b_j\right)$$

Unit -4

Communication Links:

The **directed edges (arrows)** connecting nodes define the **path of information flow**.

They:
Indicate **direction** of data (from input → output).

Carry **no weight themselves** (weights belong to nodes).

Represent **dependencies** between neurons.

$$h_1 = f(w_{11}x_1 + w_{12}x_2 + b_1)$$

$$h_2 = f(w_{21}x_1 + w_{22}x_2 + b_2)$$

$$y = f(v_1h_1 + v_2h_2 + b_3)$$

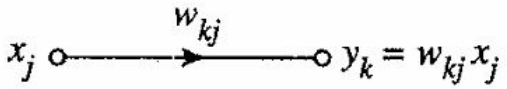
Unit -4

Neural Networks as Directed Graphs – Signal Flow Diagrams

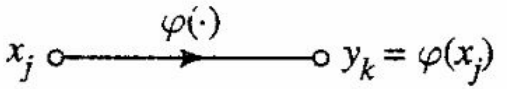
Rule 1. A signal flows along a link only in the direction defined by the arrow on the link.

Two different types of links may be distinguished:

- *Synaptic links*, whose behavior is governed by a *linear* input–output relation. Specifically, the node signal x_j is multiplied by the synaptic weight w_{kj} to produce the node signal y_k, as illustrated in Fig. 1.9a.
- *Activation links*, whose behavior is governed in general by a *nonlinear* input–output relation. This form of relationship is illustrated in Fig 1.9b, where $\varphi(\cdot)$ is the nonlinear activation function.



(a)



(b)

Unit -4

Neural Networks as Directed Graphs – Signal Flow Diagrams

Simple Example

Suppose:

- Inputs: $x_1 = 1$,$x_2 = 2$
- Weights: $w_1 = 0.5$,$w_2 = 0.2$
- Activation function:

Step 1 – Synaptic (Linear):

$$z = (0.5)(1) + (0.2)(2) = 0.9$$

Step 2 – Activation (Nonlinear):

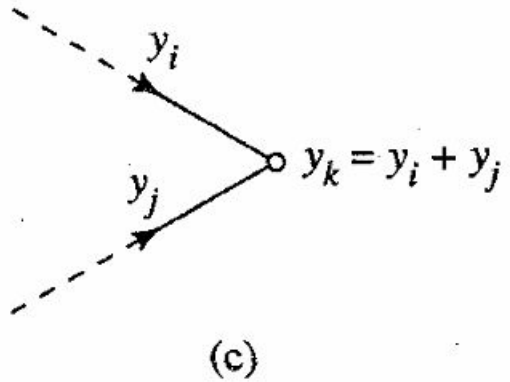
$$y = ReLU(0.9) = 0.9$$

Unit -4

Neural Networks as Directed Graphs

Rule 2. A node signal equals the algebraic sum of all signals entering the pertinent node via the incoming links.

This second rule is illustrated in Fig. 1.9c for the case of *synaptic convergence* or *fan-in*.



(c)

Unit -4

Neural Networks as Directed Graphs

Example

Let's take an example neuron with three inputs:

Input	Weight	Weighted Input
(x_1 = 1)	(w_1 = 0.4)	(0.4)
(x_2 = 2)	(w_2 = 0.6)	(1.2)
(x_3 = -1)	(w_3 = 0.5)	(-0.5)

Now sum all weighted inputs:

$$v = 0.4 + 1.2 - 0.5 = 1.1$$

Then, apply an activation function (say ReLU): y=max(0,1.1)=1.1

Unit -4

Neural Networks as Directed Graphs

Example

Suppose a neuron produces output $y = 0.8$ and sends it to three neurons via different weights:

Outgoing Link	Weight	Signal Sent
To Neuron 1	0.5	(0.8 * 0.5 = 0.4)
To Neuron 2	0.2	(0.8 * 0.2 = 0.16)
To Neuron 3	1.0	(0.8 * 1.0 = 0.8)

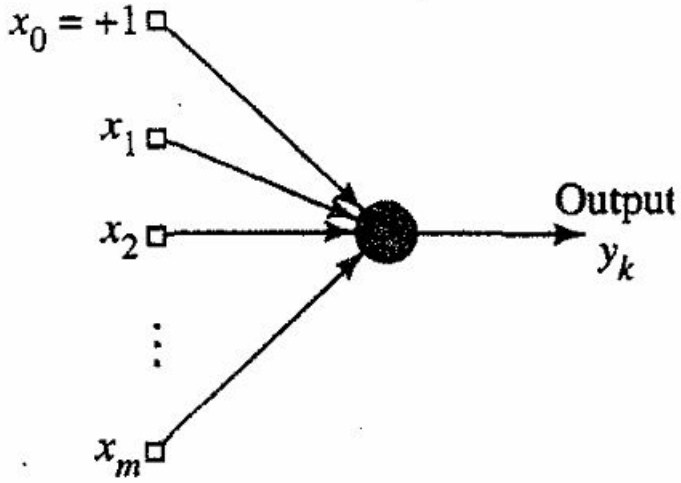
The **original output (0.8)** was transmitted to all links,

but each **receiving neuron** got a **different weighted version** of it.

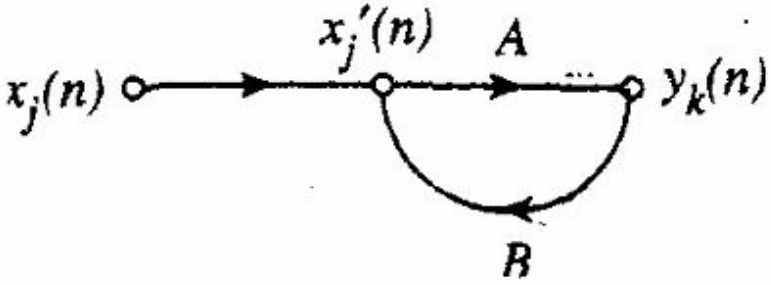
Unit -4

Feedback

Feedbacks are special class of neural networks called as Recurrent Neural Networks (RNNs)



Architectural Graph



Signal Flow Graph

Unit -4

Feedback

Example – Simple Recurrent Computation

Suppose:

$$y(n) = 0.5 \cdot x(n) + 0.4 \cdot y(n-1)$$

If:

$$x(1) = 2$$
$$y(0) = 0$$

Then:

$$y(1) = 0.5(2) + 0.4(0) = 1$$

If next input $x(2) = 1$:

$$y(2) = 0.5(1) + 0.4(1) = 0.9$$

→The output at time *n* depends on both *current* and *previous* values.

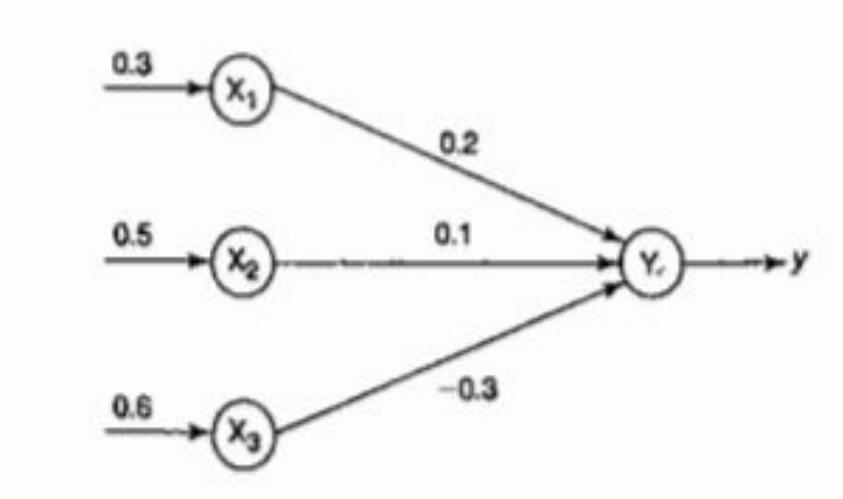
Unit -4

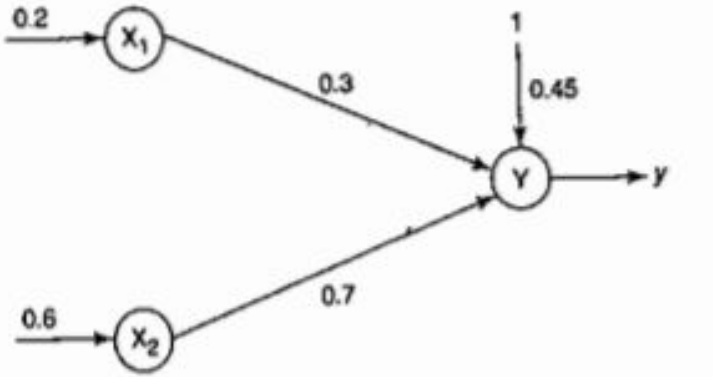
Concept	Feedforward Network	Feedback (Recurrent) Network
Flow Direction	Only forward	Forward + backward loop
Memory	No memory	Has memory of previous outputs
Graph Type	Directed acyclic	Directed cyclic (loop exists)
Equation	(y = f(Wx))	(y(n) = f(Ax(n) + Ry(n-1)))
Applications	Classification, regression	Speech, text, time-series, control

Unit -4

Problems

- For the given network, calculate the net input to the output neuron



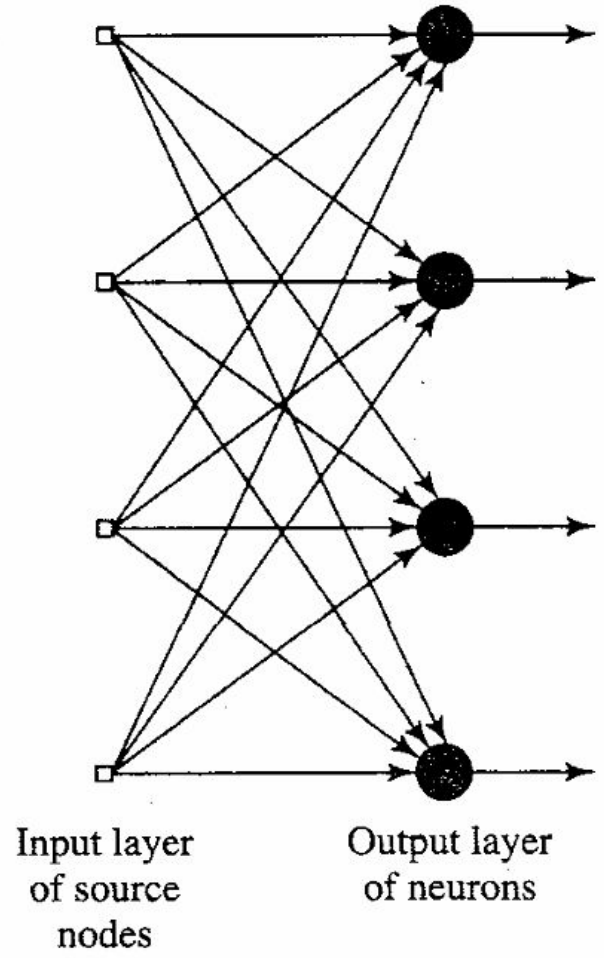


- Calculate the net input (i) Without bias and (ii) when the bias is 0.45 and observe the difference and discuss

Unit -4

Network Architeures

- Single Layer Feedforward network
  - Consists of input layer of source nodes that projects to output layer of computation node
  - Strictly feedforward or acylic type
  - Single output layer of computation nodes
- Layer is formed by taking a processing element and combining it with other processing elements
- Interconnections lead to the formation of various network architectures



Unit -4

Problem 1: Single-Layer Feedforward Neural Network: **Given:** A single neuron receives three inputs:

Input	(x_i)	Weight (w_i)
(x_1)	0.5	0.4
(x_2)	0.2	0.3
(x_3)	0.8	0.6

Bias $b = 0.1$

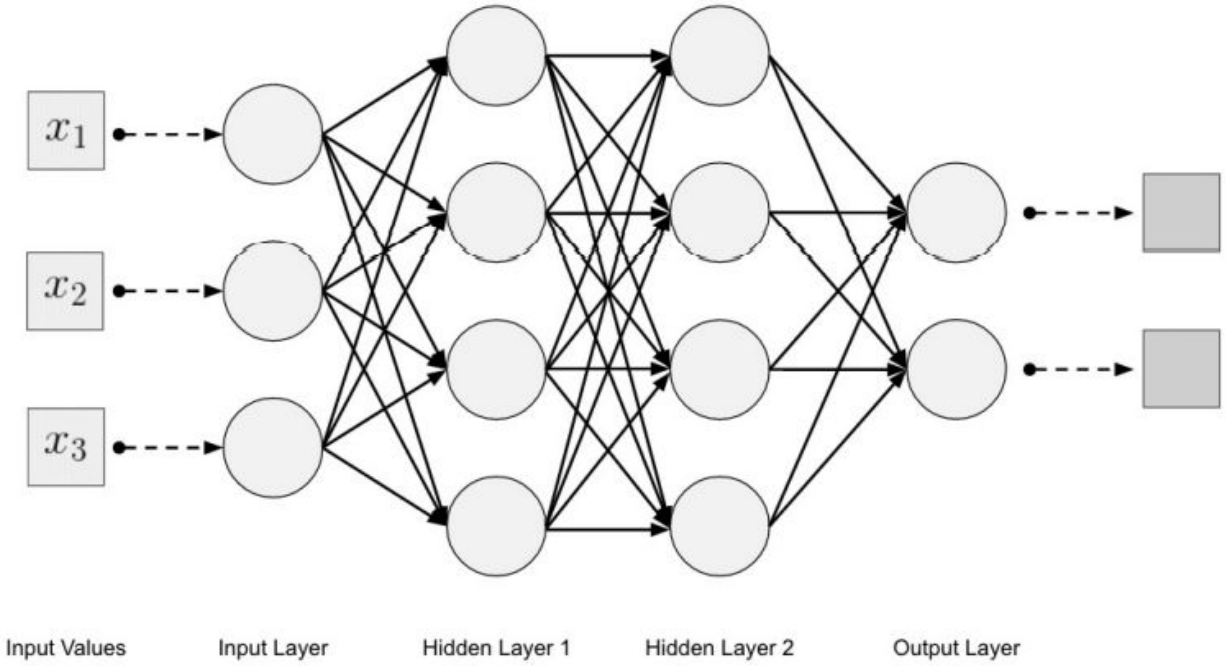
Activation Function:

$$f(v) = \frac{1}{1 + e^{-v}} \text{ (Sigmoid function)}$$

Unit -4

Network Architeures

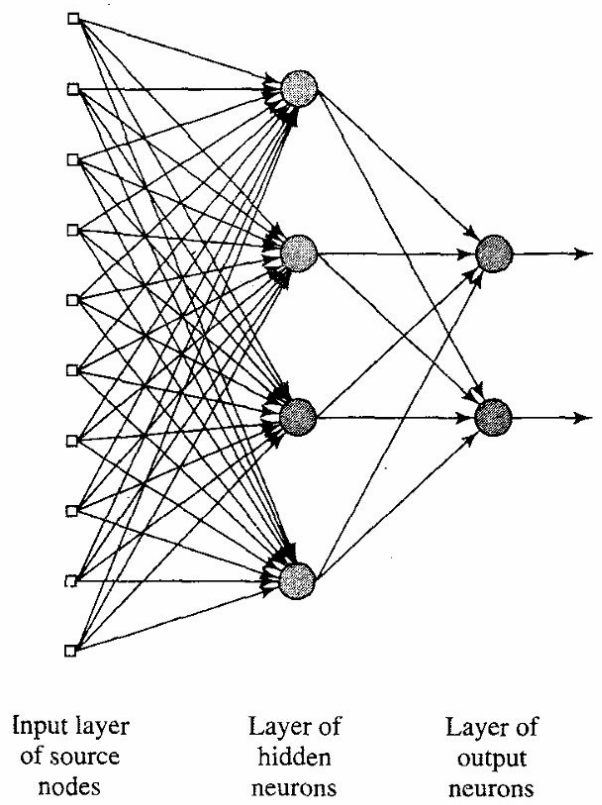
- Multi Layer Feedforward network



Unit -4

Network Architeures

- Multi Layer Feedforward network
  - Consists of input layer , hidden layer and output layer
- Each layer can have a different number of neurons and each layer is fully connected to the adjacent layer.
- The connections between the neurons in the layers form an acyclic graph



Unit -4

Problem 2: Multi-Layer Feedforward Neural Network: Given: A two-layer network (1 hidden neuron and 1 output neuron)

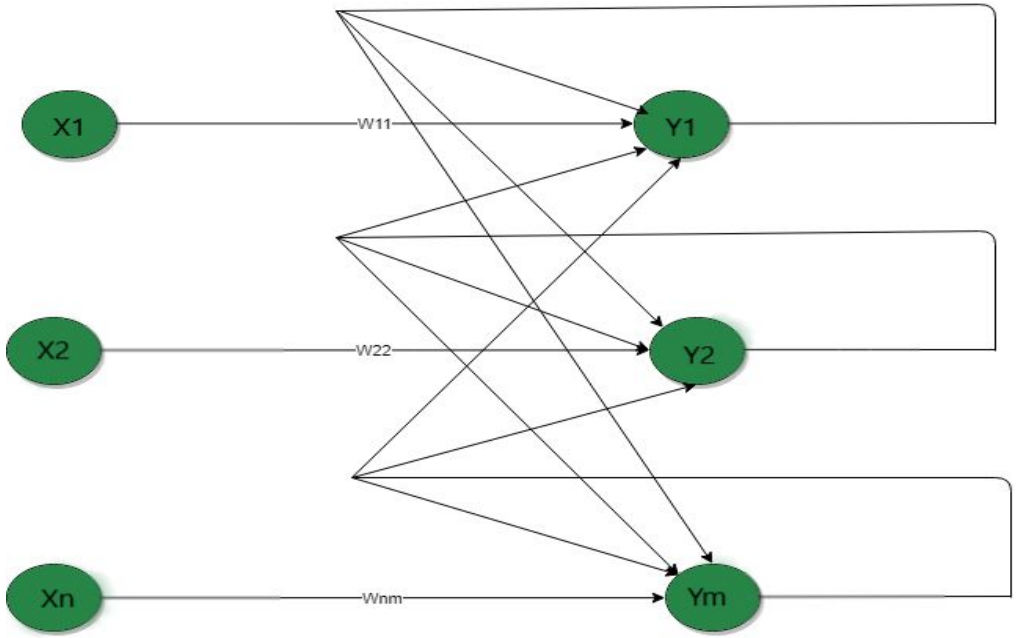
Layer	Inputs	Weights	Bias
Hidden Neuron (h_1)	(x_1=0.4, x_2=0.6)	(w_{11}=0.5, w_{12}=0.7)	(b_1=0.2)
Output Neuron (y)	(h_1)	(w_{21}=0.9)	(b_2=0.1)

Activation: Sigmoid for both layers.

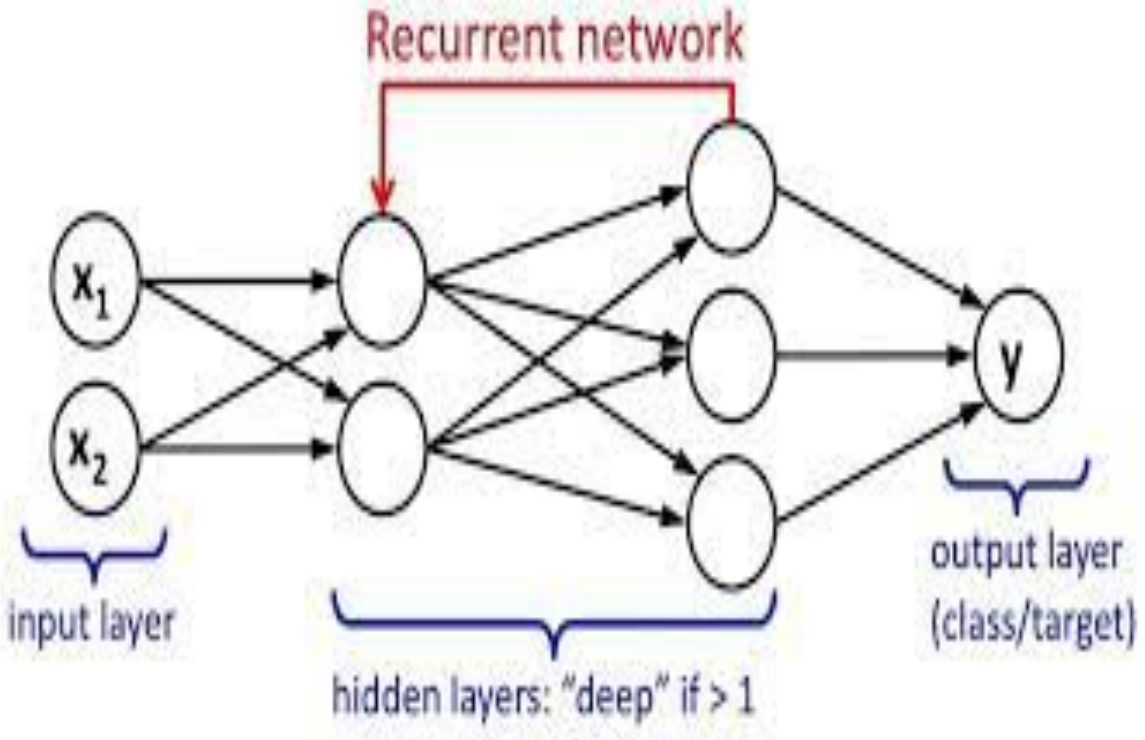
Unit -4

Recurrent Neural Network(RNN) - the output from the previous step is fed as input to the current step.

Single Layer Recurrent Network

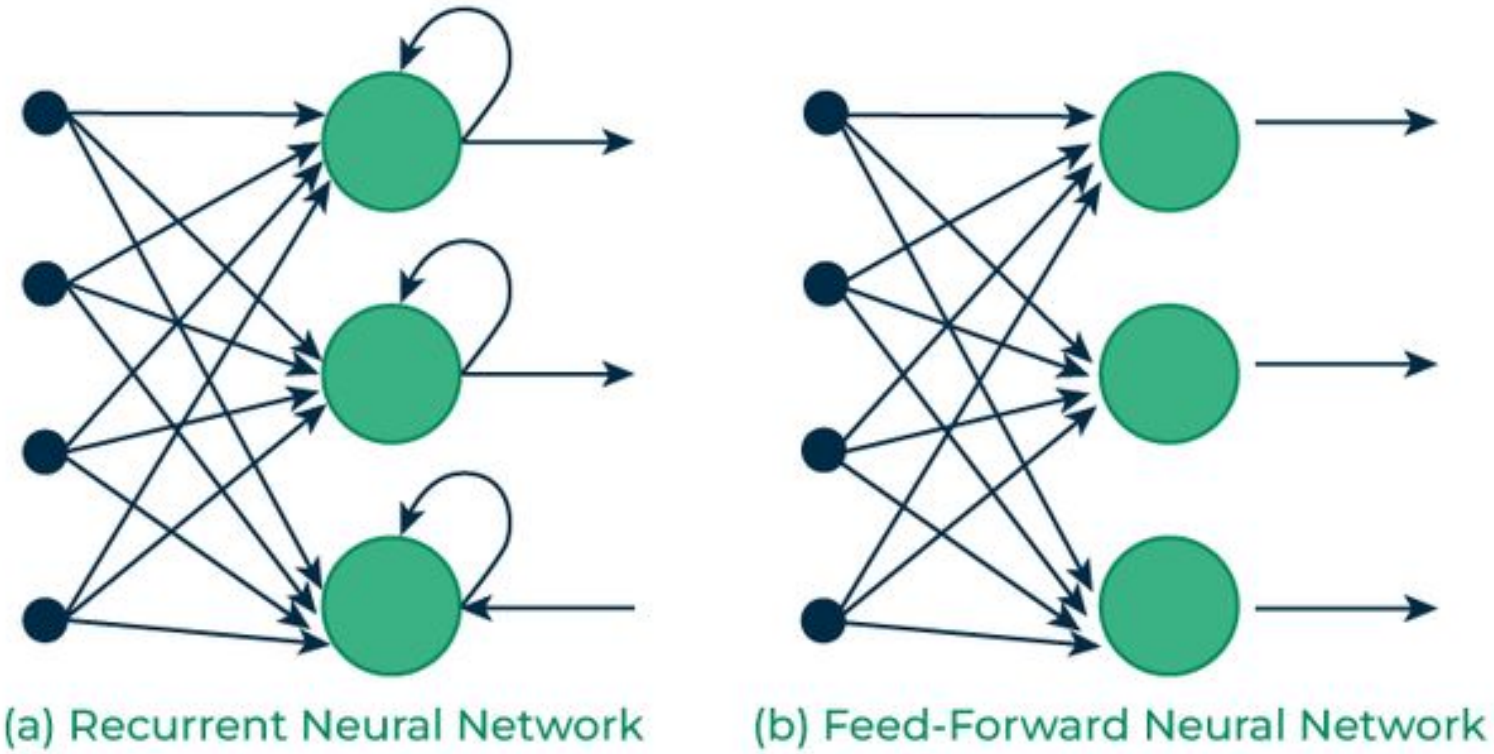


Multi Layer Recurrent Network



Unit -4

Recurrent Neural Networks Vs Feed forward Networks



(a) Recurrent Neural Network

(b) Feed-Forward Neural Network

Unit -4

Numerical Example: Backpropagation in a 2–1 Network: Network Structure

We'll use a small **feedforward neural network**:

- Inputs: $x_1 = 0.05,\ x_2 = 0.10$
- Weights: $w_1 = 0.15,\ w_2 = 0.20$
- Bias: $b = 0.35$
- Target output (desired): $t = 0.01$
- Learning rate (η): 0.5
- Activation function: Sigmoid $f(v) = \frac{1}{1+e^{-v}}$

Forward Propagation of Activity

- Step 1: Initialise weights at random, choose a learning rate η
- Until network is trained:
- For each training example i.e. input pattern and target output(s):
- Step 2: Do forward pass through net (with fixed weights) to produce output(s)
  - i.e., in Forward Direction, layer by layer:
    - Inputs applied
    - Multiplied by weights
    - Summed
    - ‘Squashed’ by sigmoid activation function
    - Output passed to each neuron in next layer
  - Repeat above until network output(s) produced

Step 3. Back-propagation of error

- Compute error (delta or local gradient) for each output unit δk
- Layer-by-layer, compute error (delta or local gradient) for each hidden unit δj by backpropagating errors (as shown previously)

Step 4: Next, update all the weights Δwij
By gradient descent, and go back to Step 2

- The overall MLP learning algorithm, involving forward pass and backpropagation of error (until the network training completion), is known as the Generalised Delta Rule (GDR), or more commonly, the Back Propagation (BP) algorithm

'Back-prop' algorithm summary

- Initialise weights at random, choose a learning rate η
- Until network is trained:
  - For each training example (input pattern and target outputs):
  - Do forward pass through net (with fixed weights) to produce outputs
    -assuming J hidden layer nodes and N inputs for a 2-layer MLP:
    $y_k = f(\sum_{j=0} w_{jk} o_j)$ where o_j is output from each hidden node j: $o_j = f\left(\sum_{i=0}^{N} w_{ij} x_i\right)$
  - For each output unit k, compute deltas: $\delta_k = (y_{target_k} - y_k) y_k (1 - y_k)$
  - For hidden units j (from last to first hidden layer, for the case of more than 1 hidden layer) compute deltas: $\delta_j = o_j (1 - o_j) \sum_k w_{jk} \delta_k$
  - For all weights, change weight by gradient descent: $\Delta w_{ij} = \eta \delta_j y_i$
    -Specifically, for the 2-layer MLP, for weight from input layer unit i to hidden layer unit j, the weight changes by: $\Delta w_{ij} = \eta \delta_j x_i$
    And, for weight from hidden layer unit j to output layer unit k, weight changes $\Delta w_{jk} = \eta \delta_k o_j$

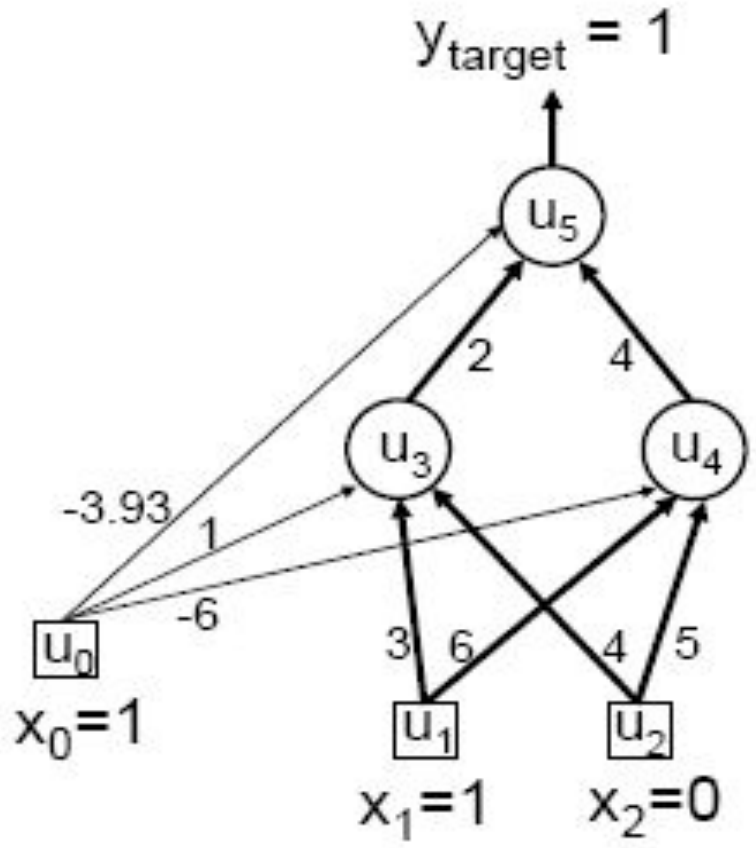
'Back-prop' algorithm summary

- Initialise weights at random, choose a learning rate η
- Until network is trained:
  - For each training example (input pattern and target outputs):
  - Do forward pass through net (with fixed weights) to produce network outputs
  - For each output unit *k*, compute deltas (local gradients):
  - For hidden units *j* (from last to first hidden layer, for the case of more than 1 hidden layer) compute deltas (local gradients):
  - For all weights, change weight by gradient descent (generalized Delta Rule):

$$\Delta w_{jk} = \eta \delta_k o_j$$

where o_j is the input to the neuron *k*, and η is the learning rate, and δ_k is local gradient for the neuron

MLP/BP: A worked example



Current state:

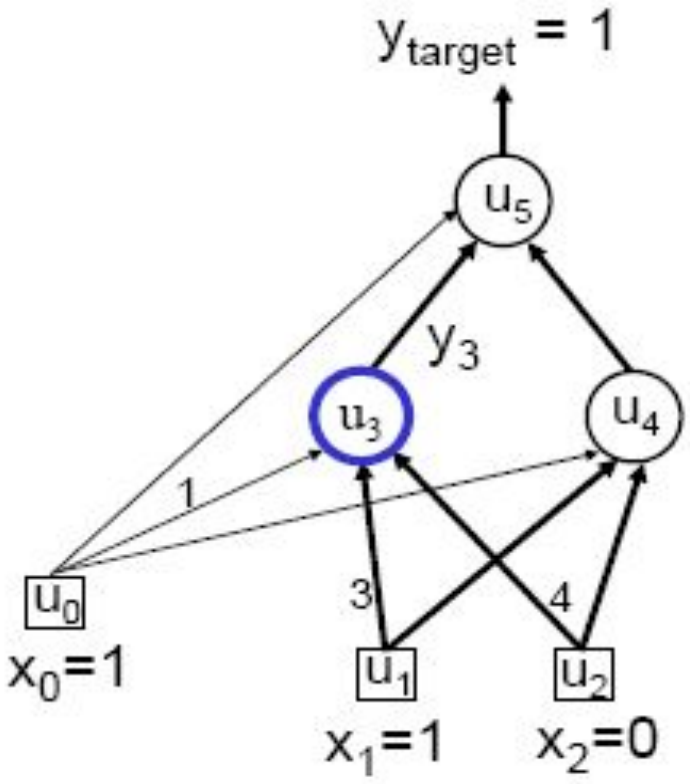
- Weights on arrows e.g. $w_{13} = 3$, $w_{35} = 2$, $w_{24} = 5$
- Bias weights, e.g.

bias for unit 4 (u_4) is w_{04}= -6

Training example (e.g. for logical OR problem):

- Input pattern is x_1=1, x_2=0
- Target output is y_{target}=1

Worked example: Forward Pass



Output for any neuron/unit j can be calculated from:

$$a_j = \sum_i w_{ij} x_i$$

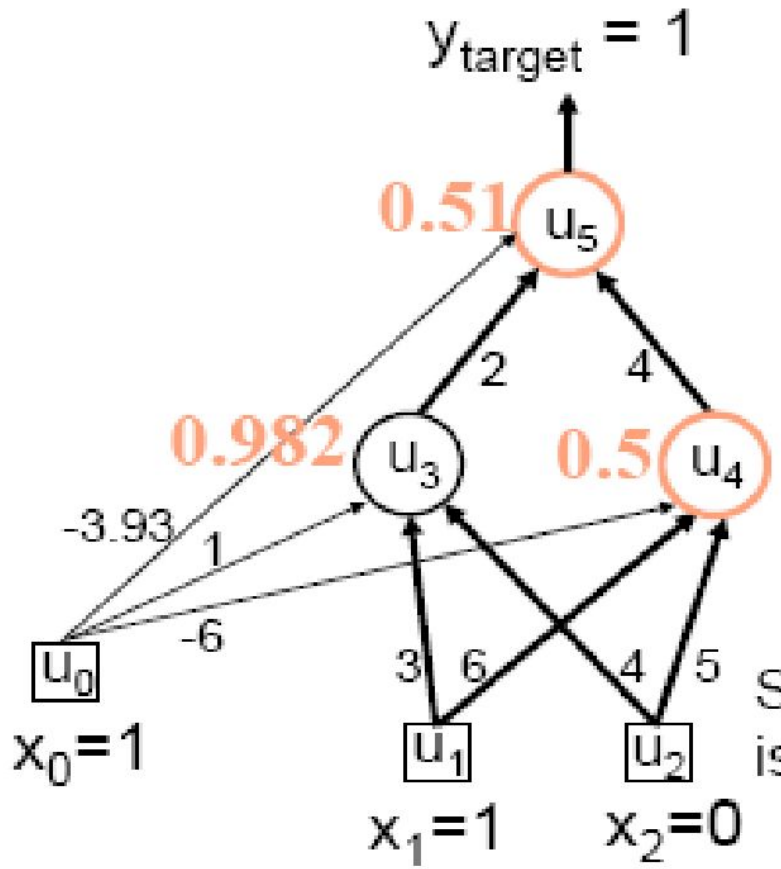
$$y_j = f(a_j) = \frac{1}{1+e^{-a_j}}$$

e.g Calculating output for Neuron/unit 3 in hidden layer:

$a_3 = 1*1 + 3*1 + 4*0 = 4$

$y_3 = f(4) = \frac{1}{1+e^{-4}} = 0.982$

Worked example: Forward Pass

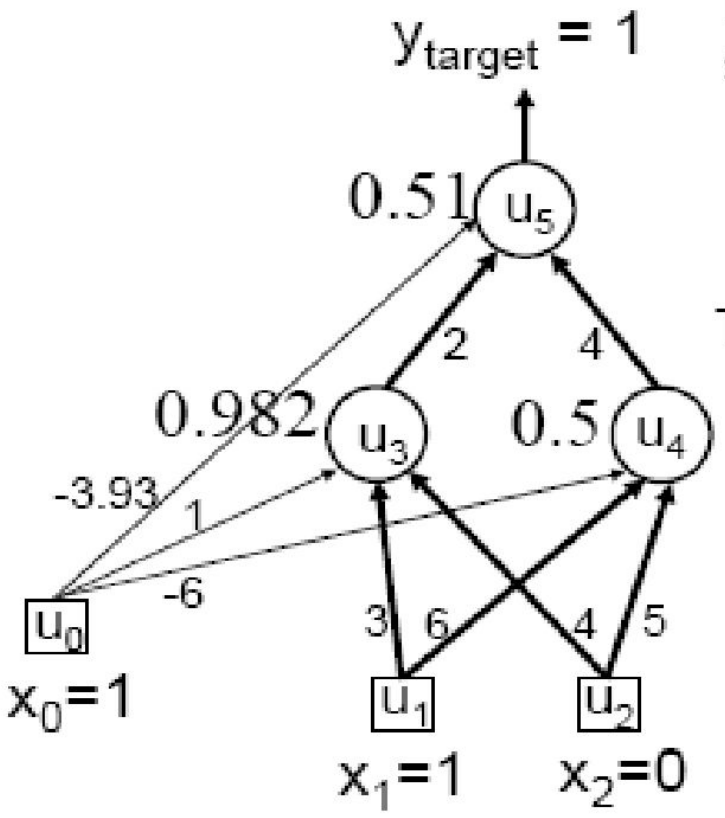


Unit	activation a_j	output y_j
u_3	4.00	0.982
u_4	0.00	0.500
u_5	0.04	**0.510**

(network output)

So the error for this training example is: $(y_{target} - y_5) = (1 - 0.510) = 0.490$

Worked example: Backward Pass



Now compute delta values starting at the output:

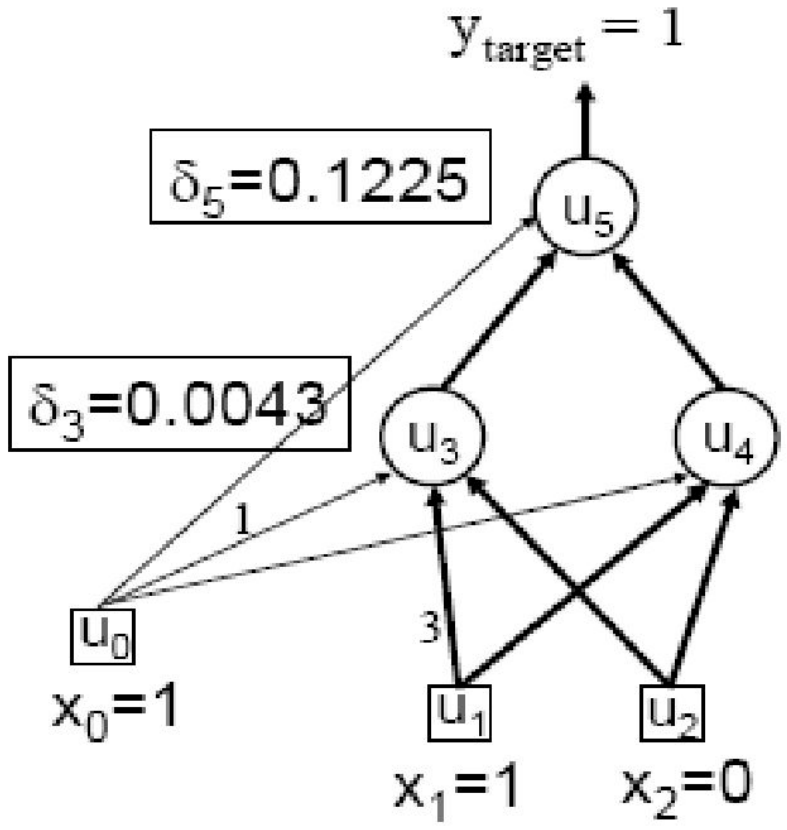
$\delta_5 = y_5(1 - y_5)(y_{target} - y_5)$
$= 0.51(1 - 0.51) \times 0.49$
$= \mathbf{0.1225}$

Then for hidden units:

$\delta_4 = y_4(1 - y_4)\, w_{45}\, \delta_5$
$= 0.5(1 - 0.5) \times 4 \times 0.1225$
$= \mathbf{0.1225}$

$\delta_3 = y_3(1 - y_3)\, w_{35}\, \delta_5$
$= 0.982(1-0.982) \times 2 \times 0.1225$
$= \mathbf{0.0043}$

Worked example: Update Weights Using Generalized Delta Rule (BP)



- Set learning rate η = 0.1

  Change weights by:

  $$\Delta w_{ij} = \eta \delta_j y_i$$

- e.g.bias weight on u_3:

$\Delta w_{03} = \eta \delta_3 x_0$

$=0.1*0.0043*1$

$= 0.0004$

So, new w_{03} ⌫

$w_{03}(old)+\Delta w_{03}$

$=1+0.0004=1.0004$

- and likewise:

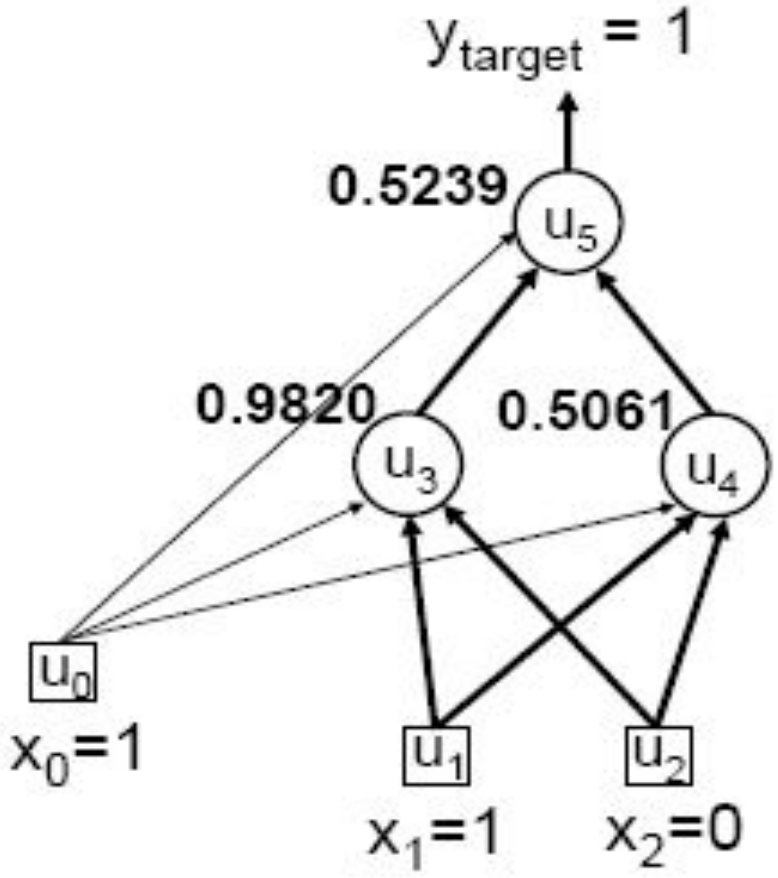
  w_{13} ⌫ 3 + 0.0004

  = 3.0004

Similarly for the all weights wij:

i	j	w_{ij}	δ_j	y_i	Updated w_{ij}
0	3	**1**	0.0043	1.0	**1.0004**
1	3	**3**	0.0043	1.0	**3.0004**
2	3	**4**	0.0043	0.0	**4.0000**
0	4	**-6**	0.1225	1.0	**-5.9878**
1	4	**6**	0.1225	1.0	**6.0123**
2	4	**5**	0.1225	0.0	**5.0000**
0	5	**-3.92**	0.1225	1.0	**-3.9078**
3	5	**2**	0.1225	0.9820	**2.0120**
4	5	**4**	0.1225	0.5	**4.0061**

Verification that it works



On next forward pass:

The new activations are:

$y_3 = f(4.0008) = 0.9820$

$y_4 = f(0.0245) = 0.5061$

$y_5 = f(0.0955) = \mathbf{0.5239}$

Thus the new error

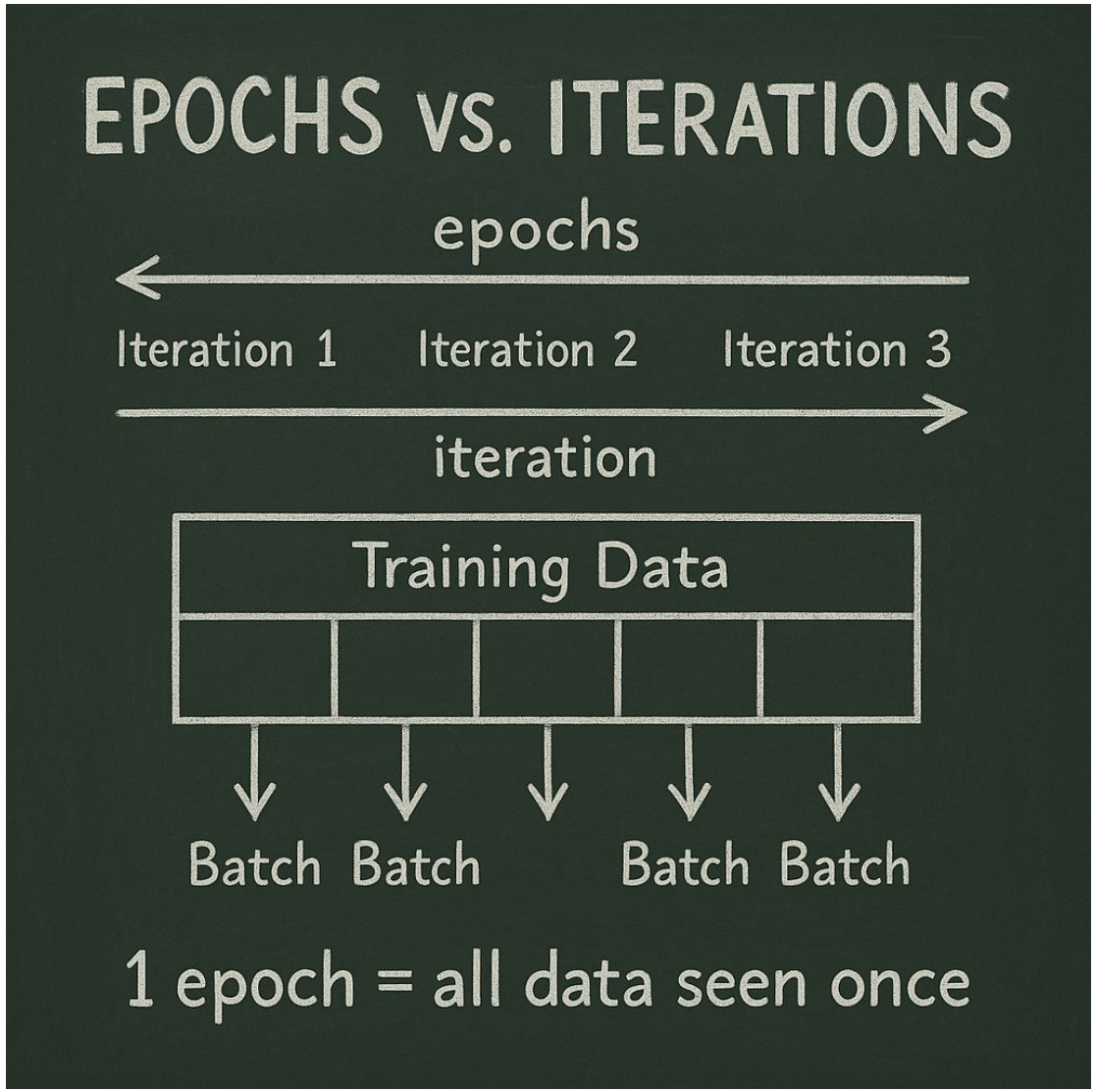
$(y_{target} - y_5) = (1 - 0.5239) = 0.476$

has been reduced by **0.014**

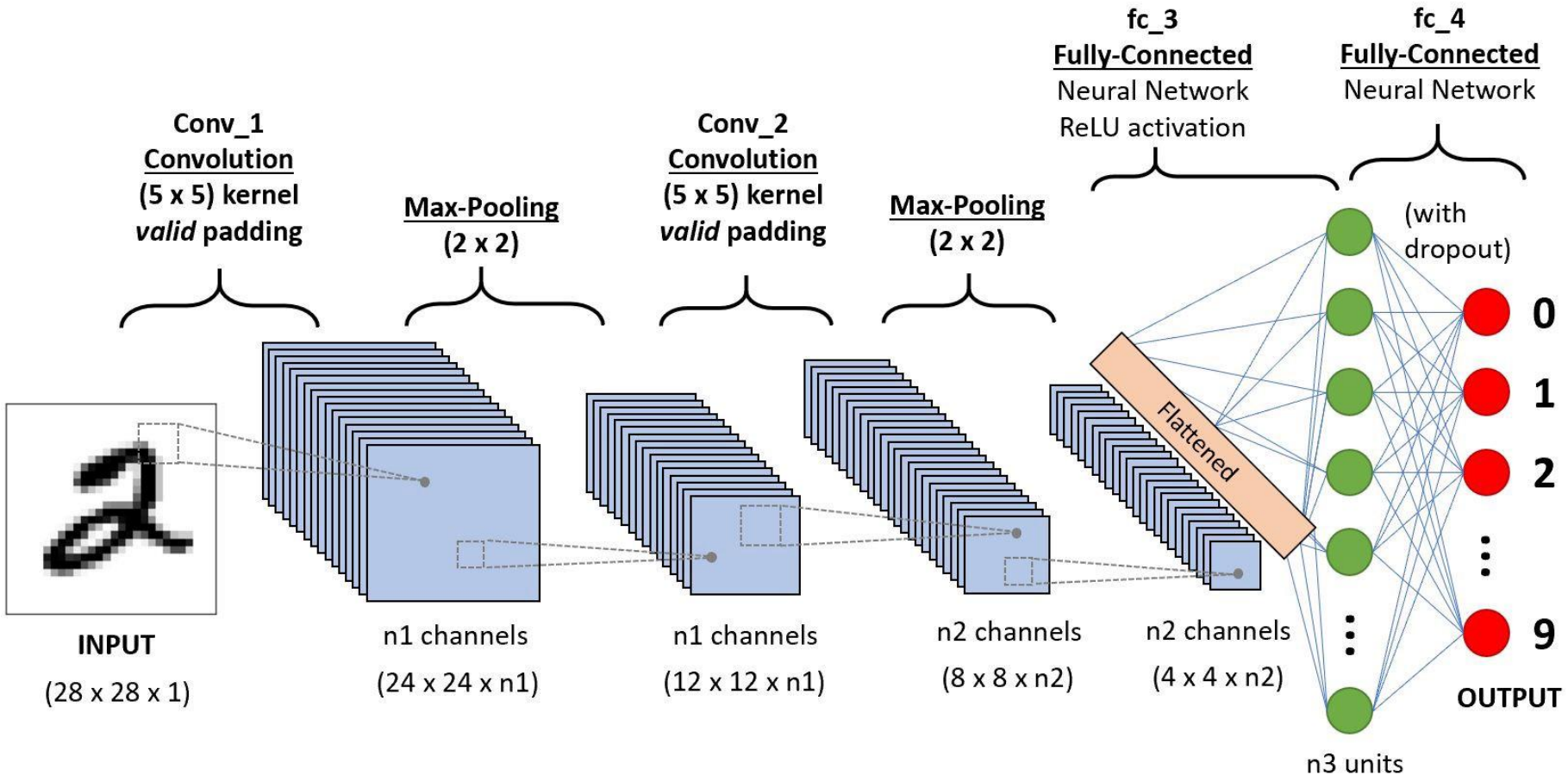
(from **0.490** to **0.476**)

Ref: "Neural Network Learning & Expert Systems" by Stephen Gallant

Iterations vs Epochs



Structure of CNN



Structure of CNN

Term	Meaning (how to tell in class)
Input size ($H_{\square} \times W_{\square}$)	"This is the height and width of the image we give to CNN." Example: 28×28 image
Kernel size (K × K)	"This is the small box (filter) that slides over the image to detect features." Common sizes: 3×3 or 5×5
Stride (S)	"How many steps the filter moves at a time." Stride 1: move 1 pixel at a time (slow, detailed) Stride 2: jump 2 pixels (faster, smaller output)
Padding (P)	"Adding extra border pixels around the image so that corner data is not lost and size can be controlled." Zero padding means we add 0s around the border
Output size ($H_o \times W_o$)	"The final size of the feature map after the filter finishes sliding over the image."

Structure of CNN

Given:

Input = 7×7 image

Kernel = 3×3

Stride = 1

Padding = 0

$$Output = \frac{7 - 3 + 2(0)}{1} + 1 = 5$$

So the feature map will be **5 × 5**

- Input = 7
- Kernel = 3
- Stride = 1
- Padding = 1

$$Output = \frac{7 - 3 + 2(1)}{1} + 1 = 7$$

Structure of CNN

Pool1 (Max-Pooling 2×2, stride 2)

Pooling *reduces height and width*, formula:

$$H_{out} = \frac{H_{in}}{S}$$

Parameter	Value
(H_{in})	24
Stride	2

H_out=24/2=12

Output after Pool1: **12 × 12 × n_1**

Structure of CNN

Conv2 (5×5 kernel, stride 1, VALID padding)

Apply convolution formula again:

$$H_{out} = \frac{(12 - 5 + 2(0))}{1} + 1 = 8$$

Output after Conv2: **8 × 8 × n_2….**

(n_2 = number of filters in Conv2, so depth increases)

Pool2 (Max-Pooling 2×2, stride 2)

$$H_{out} = \frac{8}{2} = 4$$

Output after Pool2: **4 × 4 × n_2**

Flatten

Flattening means converting the 3D tensor (height × width × depth) into a 1D vector:

$$4 \times 4 \times n_2 = 16n_2 \ units$$

Flatten result: **16 · n_2 neurons** → fed into fully connected (dense) layers

Components of CNN

- **Key points about Convolution layers and Filters**

- **The depth of a filter in a CNN must match the depth of the input image**. The number of color channels in the filter must remain the same as the input image.

- **Different Conv2D filters are created for each of the three channels** for a color image.

- **Filters for each layer are randomly initialized based on either Normal or Gaussian distribution.**

- **Early layers have fewer filters; deeper layers have more filters**
- Early layers extract *simple features*.
- Deep layers extract *complex features*.

Components of CNN

Layer Level	What filters learn	Typical # of filters
Shallow layers (Conv1, Conv2)	Edges, curves, simple patterns	8, 16, 32
Middle layers	Shapes, textures	64, 128
Deep layers	High-level features (eyes, objects, digits)	256, 512

More filters → more computation

- Deeper layers with many filters learn better features, **but require more computation and memory.**

Consider the convolutional neural network defined by the layers in the left column below. Fill in the shape of the output volume and the number of parameters at each layer. You can write the shapes in the NumPy format (e.g. (128,128,3)).

Notation:

- CONV5-N denotes a convolutional layer with N filters with height and width equal to 5. Padding is 2, and stride is 1.
- POOL2 denotes a 2x2 max-pooling layer with stride of 2 and 0 padding.
- FC-N denotes a fully connected layer with N neurons.

Layer	Activation Volume Dimensions	Number of parameters
Input	32*32*1	0
CONV5-10		
POOL2		
CONV5-10		
POOL2		
FC10		

Convolution Layer

- Filters or kernels are the base units in this system.
- These layers consist of a series of learnable filters, meaning the CNN adjusts their values during training.
- Convolution is done by computing the dot product between the entries of the filter and the volume.
- Let us understand the process of convolution with an example.

(a)

0	1	1	0	0
0	0	1	0	0
0	1	1	0	0
0	0	1	0	0
0	1	1	0	0

(b)

0	1	1
0	0	1
0	1	1

Feature Map Creation

(a)

0×0	1×1	1×1	0	0
0×0	0×0	1×1	0	0
0×0	1×1	1×1	0	0
0	0	1	0	0
0	1	1	0	0

(b)

5		

(a) First subset of the input feature; (b) first value of the feature map.

(a)

0	1×0	1×1	0×1	0
0	0×0	1×0	0×1	0
0	1×0	1×1	0×1	0
0	0	1	0	0
0	1	1	0	0

(b)

5	2	

(a) Second subset of the input feature; (b) second value of the feature map.

Parameters

Term	Meaning
Input size ($H_{\square} \times W_{\square}$)	"This is the height and width of the image we give to CNN." Example: 28×28 image
Kernel size (K × K)	"This is the small box (filter) that slides over the image to detect features." Common sizes: 3×3 or 5×5
Stride (S)	"How many steps the filter moves at a time." Stride 1: move 1 pixel at a time (slow, detailed) Stride 2: jump 2 pixels (faster, smaller output)
Padding (P)	"Adding extra border pixels around the image so that corner data is not lost and size can be controlled." Zero padding means we add 0s around the border
Output size ($H_o \times W_o$)	"The final size of the feature map after the filter finishes sliding over the image."

Feature Map

- The size of the feature map (convolved feature) is controlled by three parameters (to be decided before the convolution)

- **Depth:** no. of filters, depth of feature map n
- **Stride:** filter matrix pixel slided over the input matrix
- **Zero Padding:** Adjusting the input size as per the requirement by adding zeros to the input matrix.

Padding

- **Padding in Convolutional Neural Networks (CNNs)** refers to adding extra layers of pixels around the edges of an input image or feature map.

- This technique is used to control the spatial dimensions of the output after a convolution operation.

Why Use Padding?

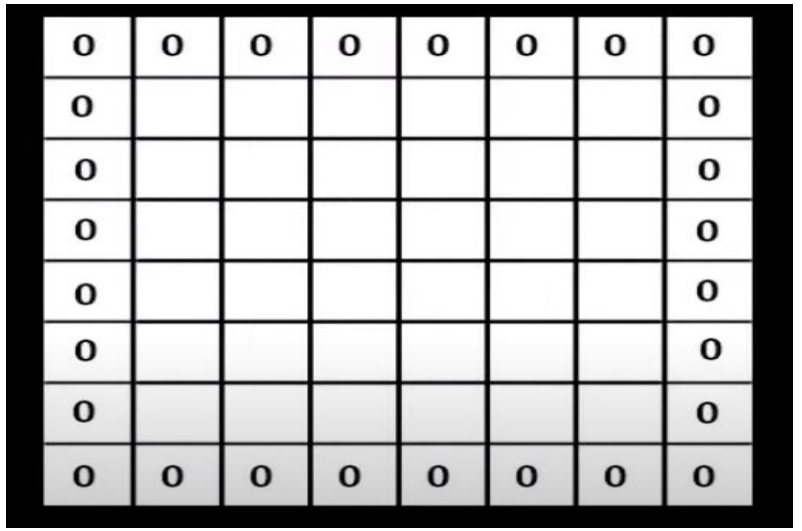
1. **Preserve Spatial Dimensions**:
    1. Without padding, applying filters reduces the dimensions of the input feature map, especially near the edges.
2. **Prevent Information Loss**:
    1. Filters inherently struggle to process the edges of an image because there are fewer surrounding pixels.
3. **Allow Deeper Networks**:
    1. Maintaining dimensions across layers (using padding) helps design deeper networks without quickly shrinking the feature maps.

Padding

Zero padding (most common): add rows/columns of zeros around the input before convolving.

Why use it

- Simple and cheap to compute.
- Preserves more spatial information near borders than "valid" (no padding).



Valid padding ("no padding")

Definition: do not add any padding; convolution is only applied where the filter entirely overlaps the input.

What it looks like:

With $K = 3$ on a 5×5 input, the filter can only be placed in positions that keep it fully inside the image → output shrinks.

Padding

1.Same Padding:

1. Adds enough padding so the output dimensions are the same as the input dimensions.
2. Ensures consistent size across layers.

Zero Padding = *The method of adding zeros around the image.*
Same Padding = *A goal/strategy that uses padding (usually zeros) so that the* ***output size stays the same as the input size****.*

Step 1: Original 5×5 Image

1	2	3	4	5
6	7	8	9	1
1	3	5	7	9
2	4	6	8	1
9	8	7	6	5

0	0	0	0	0	0	0
0	1	2	3	4	5	0
0	6	7	8	9	1	0
0	1	3	5	7	9	0
0	2	4	6	8	1	0
0	9	8	7	6	5	0
0	0	0	0	0	0	0

Step 3: Perform convolution (using 3×3 kernel)

Since we added padding, the filter can slide over **every original pixel**, including edges.

Output size formula:

$$H_{out} = \frac{H_{in} + 2P - K}{S} + 1$$

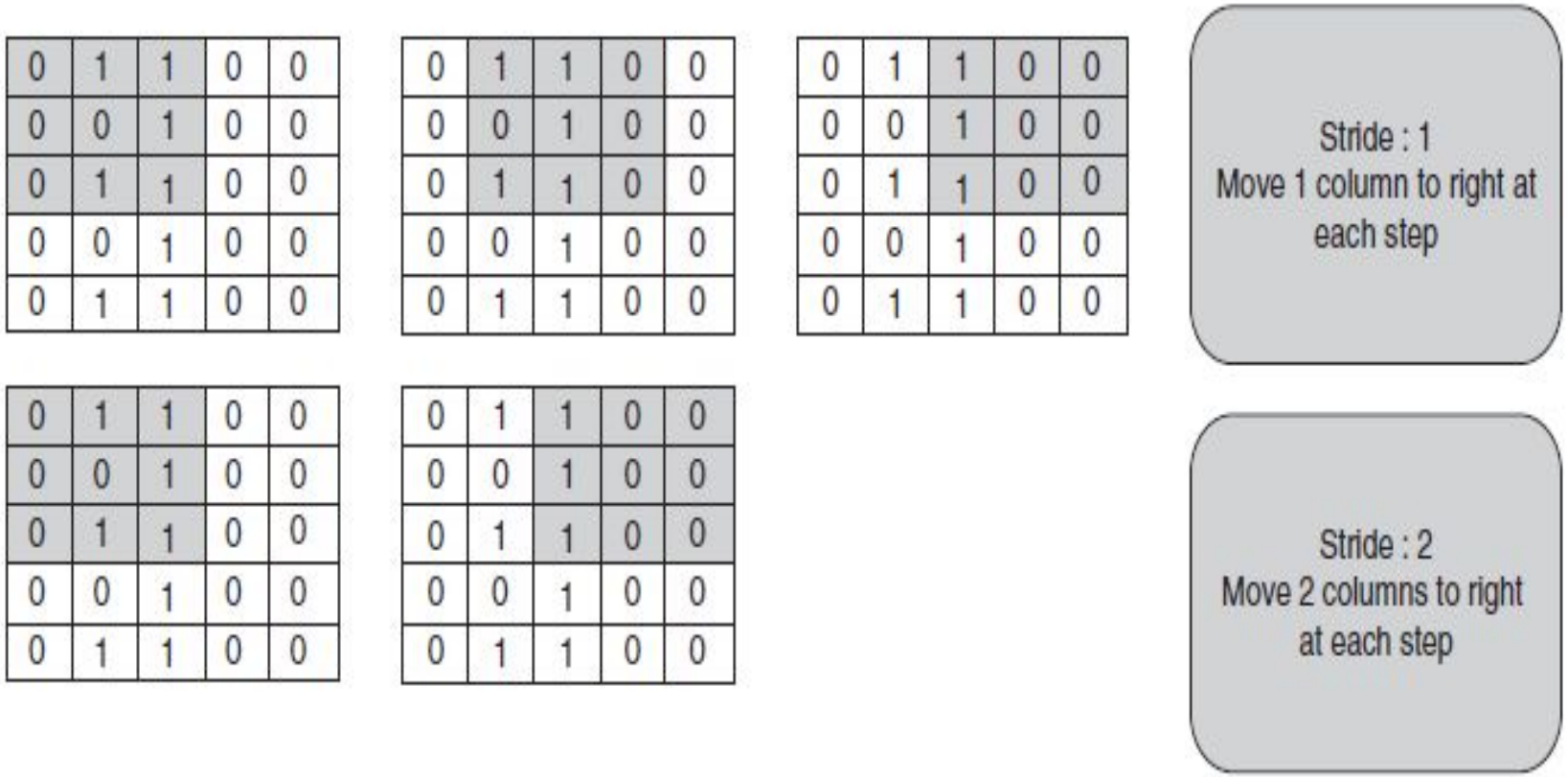
Substitute:

$$H_{out} = \frac{5 + 2(1) - 3}{1} + 1 = 5$$

Output size stays **5 × 5**

That is why it's called **SAME padding** → Same input size **=** Same output size

Representation of Stride



Example-1: Numerical Problems

- Given the input matrix and the kernel, perform convolution with stride being 1 and 2.

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input matrix

1	0	0
0	0	1
1	1	0

Kernel

- Perform convolution with stride1

Step 1:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel

2		

Feature Map

Step 2:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel

2	1	

Feature Map

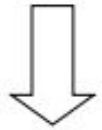
Step 3:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	1	2

Feature Map

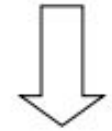
Step 4:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	1	2
1		

Feature Map

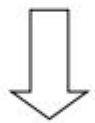
Step 5:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	1	2
1	2	

Feature Map

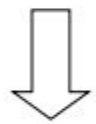
Step 6:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	1	2
1	2	3

Feature Map

Step 7:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel

2	1	2
1	2	3
4		

Feature Map

Step 8:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel

2	1	2
1	2	3
4	2	

Feature Map

Step 9:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel

2	1	2
1	2	3
4	2	1

Feature Map after applying stride1

- # Perform convolution with stride2

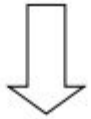
Step 1:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	

Feature Map

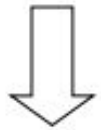
Step 2:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	2

Feature Map

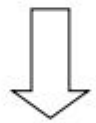
Step 3:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	2
1	

Feature Map

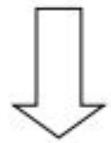
Step 4:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	2
1	3

Feature Map

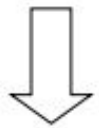
Step 5:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	2
1	3
4	

Feature Map

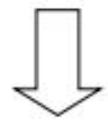
Step 6:

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input

1	0	0
0	0	1
1	1	0

Kernel



2	2
1	3
4	1

Feature Map for Stride 2

Example -2:

INPUT

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

FILTER

1	0	1
0	1	0
1	0	1

CONVOLVED FEATURE

4		

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

1	0	1
0	1	0
1	0	1

4		
2		

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

1	0	1
0	1	0
1	0	1

4	3	
2		

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

1	0	1
0	1	0
1	0	1

4	3	4
2	4	3
2	3	4

Pooling or Down sampling Layer

- Pooling Layer function is to progressively *reduce the spatial size* of the representation to reduce the number of parameters and computation in the network.
- Periodically insert pooling layer in between successive convolution layer
- Resize operation
- Hence to also control overfitting.
- Spatial pooling is called *subsampling or downsampling*.
- It reduces the dimensionality of each map but retains the important information.

Types Of Spatial Pooling

- **Max-Pooling:**

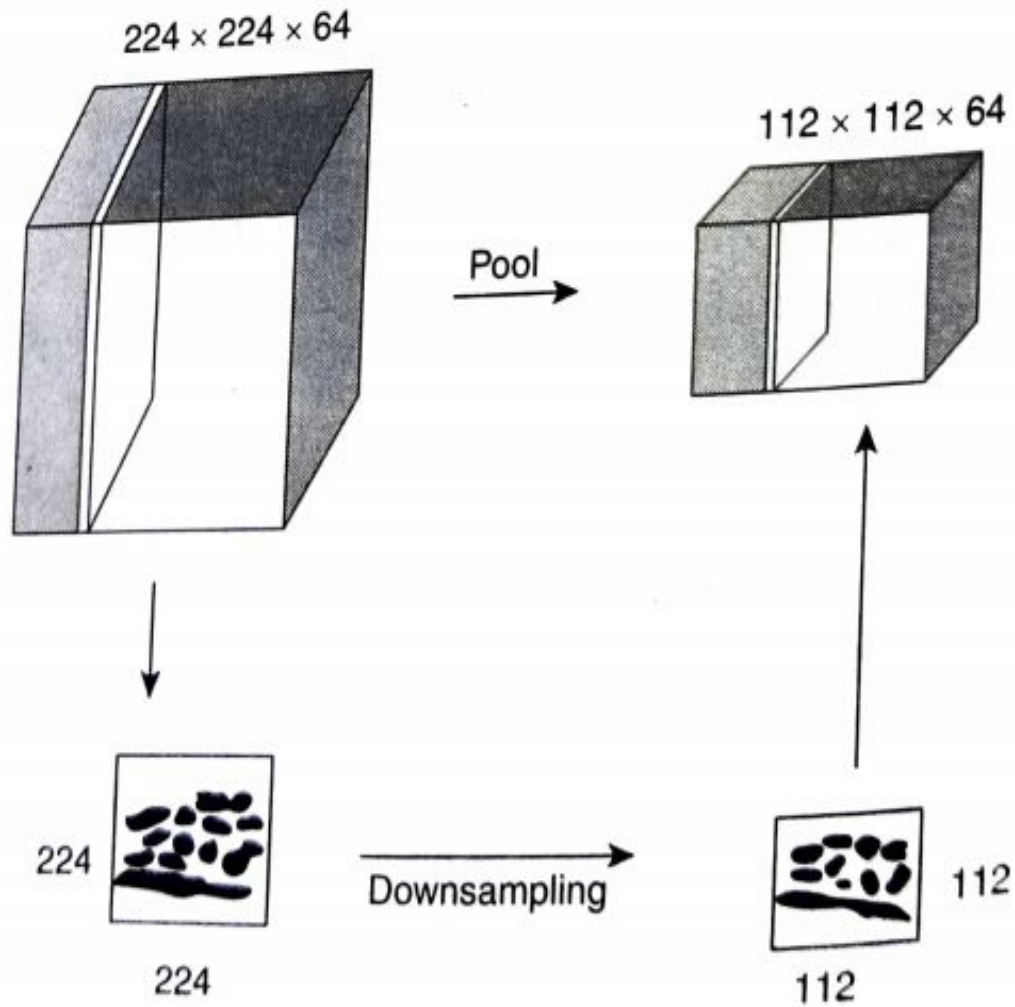
  Max-pooling takes the *largest element* from the rectified feature map.
  Ex. Sharp edges, curves

- **Average Pooling:**

  Average pooling works well for straight lines and smaller curves (smooth fashion), but cannot detect extreme features like sharp edges.

- **Sum Pooling:**

  The sum of all elements in the feature map is called sum pooling. Decreases the dimension of input data.


224 × 224 × 64
Pool
112 × 112 × 64
224
Downsampling
112
224
112

Flattening Layer

- Fully connected layers cannot process multidimensional data
- So reduced down to single dimension (flattened)

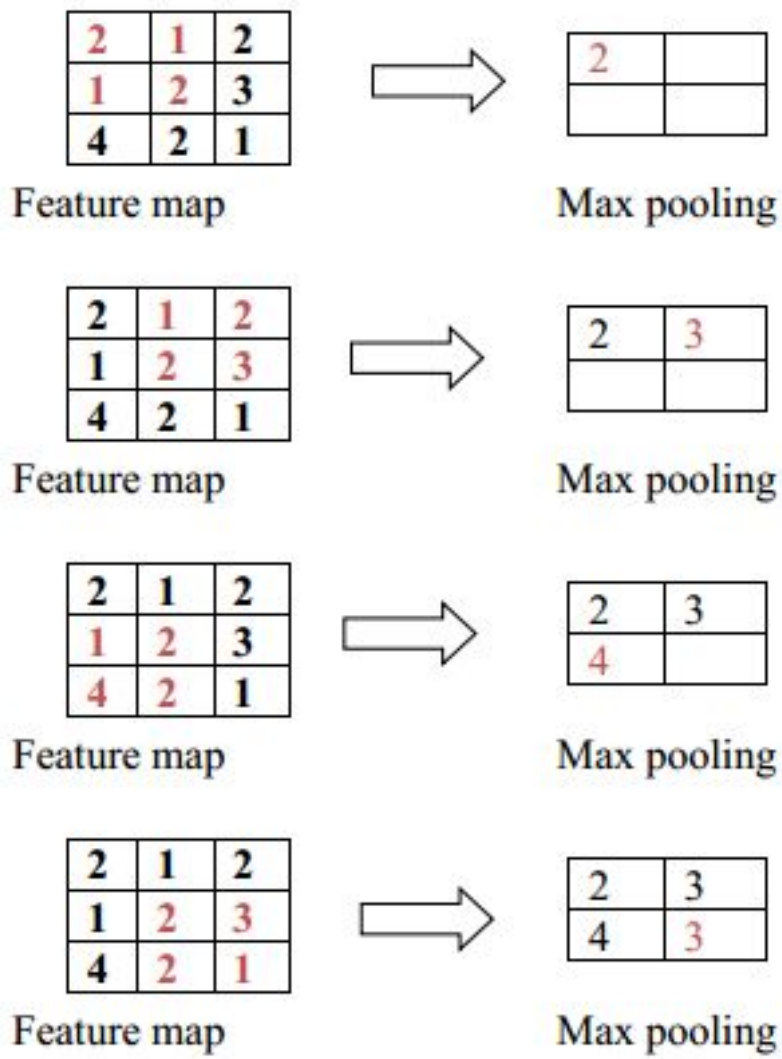
Fully connected layer

- FC layer is a traditional MLP.
- Every neuron in the previous layer is connected to every neuron on the next layer.
- Classification based on training dataset.
- Output probabilities

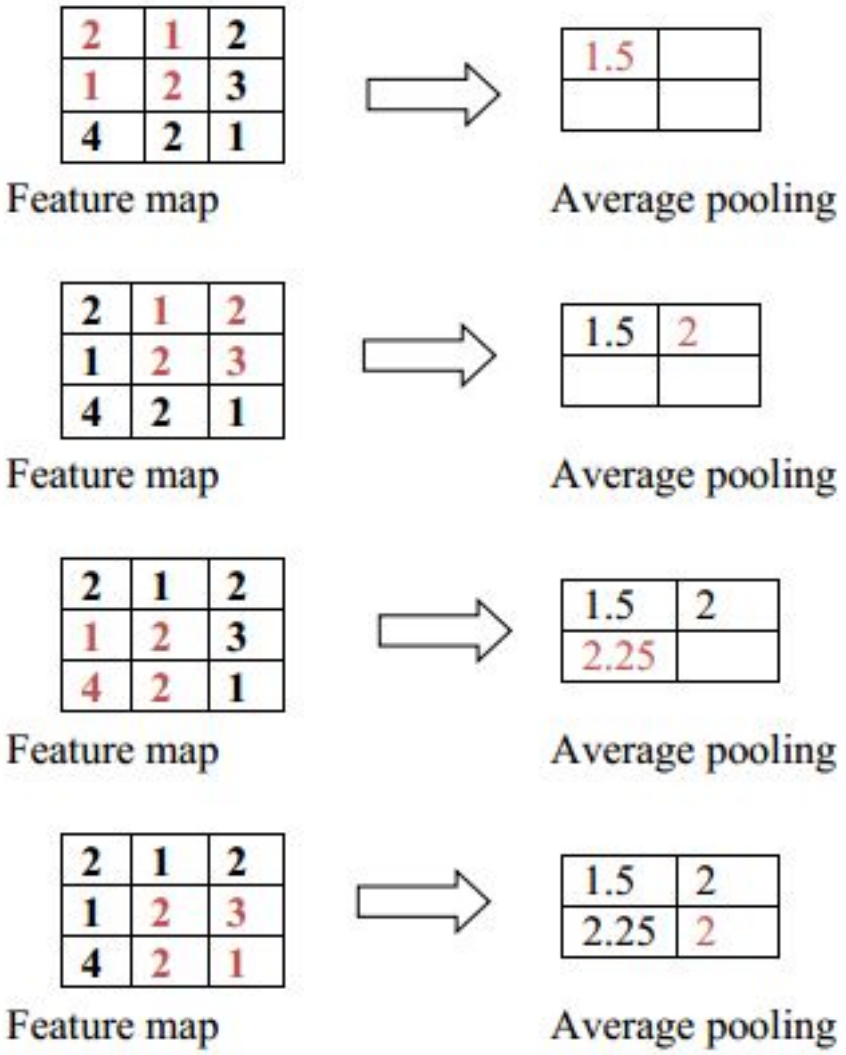
Question: Apply Max-pooling, Average-pooling, and Sum-pooling to the results of convolutions (Example-1)

Feature map after applying stride1.

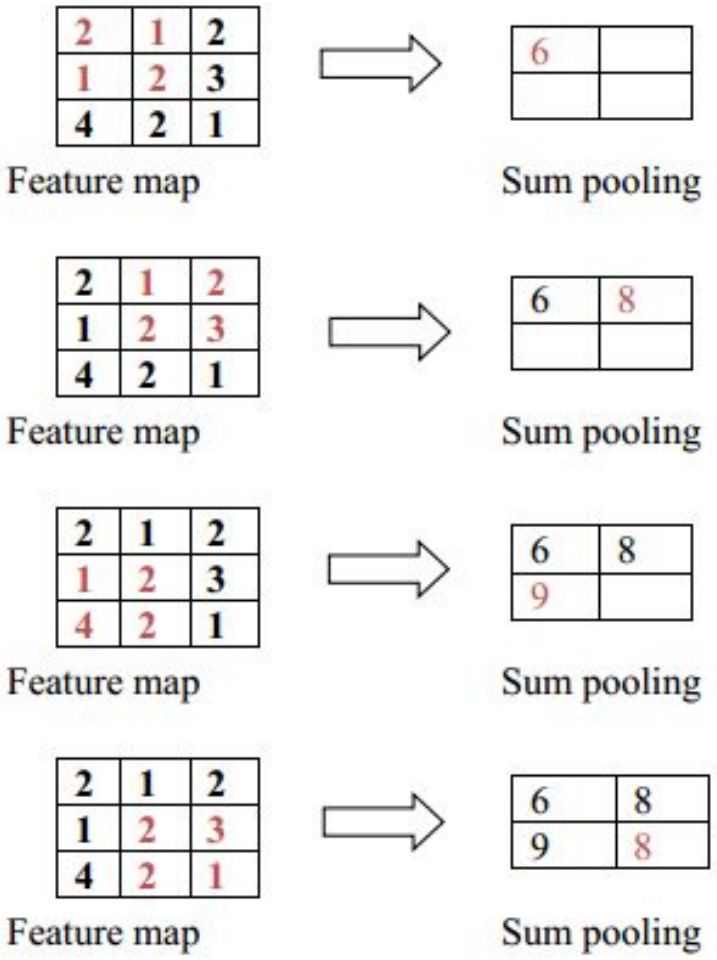
- Apply max pooling



- Apply average pooling



- Apply Sum pooling



- flattened version of the pooled feature maps

Max Pooling:

2	3
4	3

→

2
3
4
3

Average Pooling:

1.5	2
2.25	2

→

1.5
2
2.25
2

Sum Pooling:

6	8
9	8

→

6
8
9
8

Introduction and expressiveness of RNN.

Why RNNs? (Motivation)

- Traditional feedforward networks cannot model:
  - Sequential patterns – Ordered Relationship, one after the other
  - Time-series dependencies – Current value depending on the past
  - Language structure – Rules and patterns – on how words and sentences are formed

- Many real-world sequences:
  - Speech, text, DNA sequences
  - Machine sensor data, video frames

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Introduction and expressiveness of RNN.

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Introduction and expressiveness of RNN.

Expressiveness of RNN.

- RNNs are **Turing complete** (Siegelmann & Sontag)

Can model any computable function with:

- Sufficient hidden units
- Proper training

Capable of learning:

- Context - previous words, events, or data points that help understand the current one.

- Long-range dependencies – long sentences, events over the time

- Grammar-like structures

Basic RNN Model

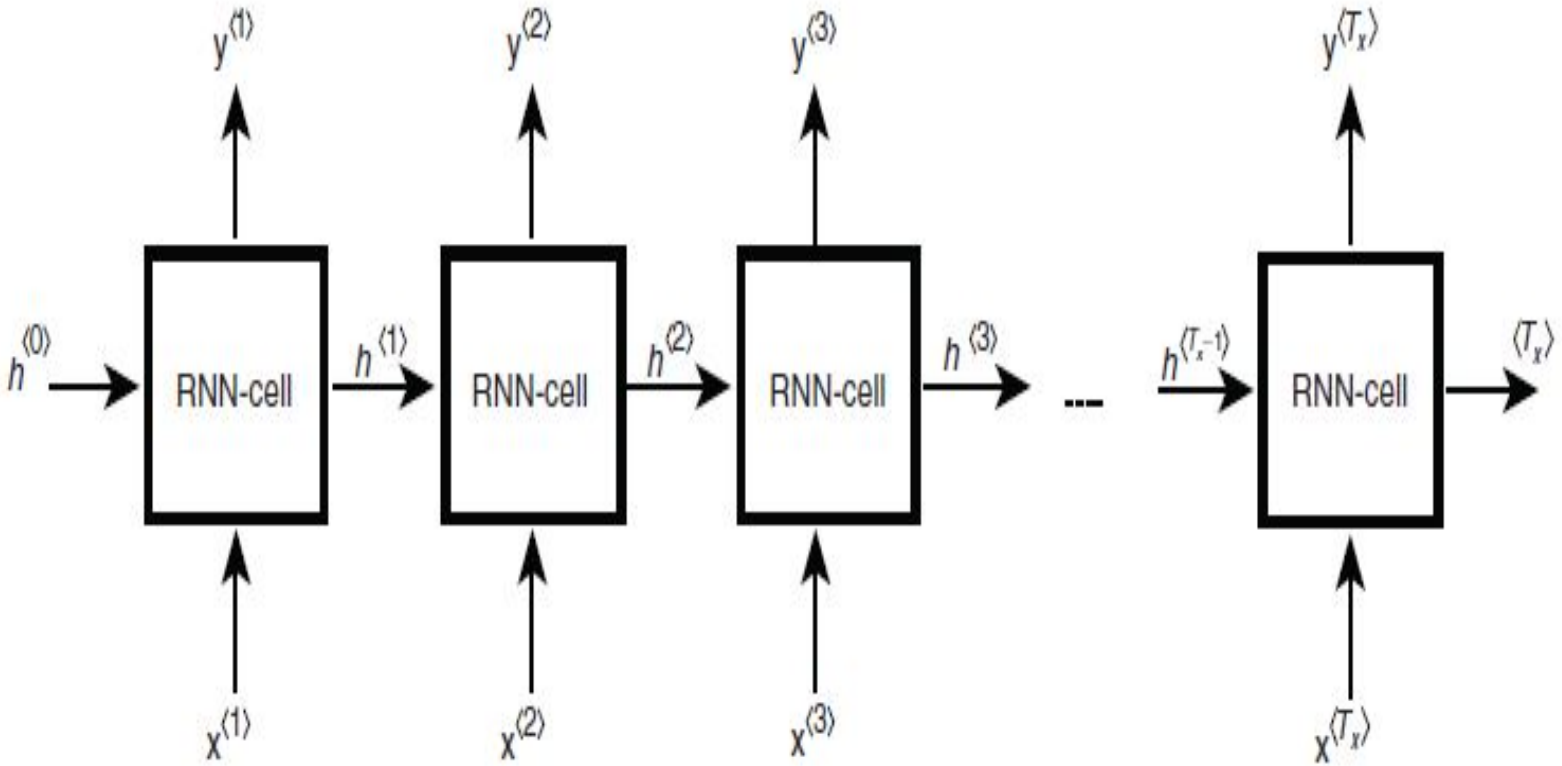


Fig. Basic RNN Model

Structure of RNN.

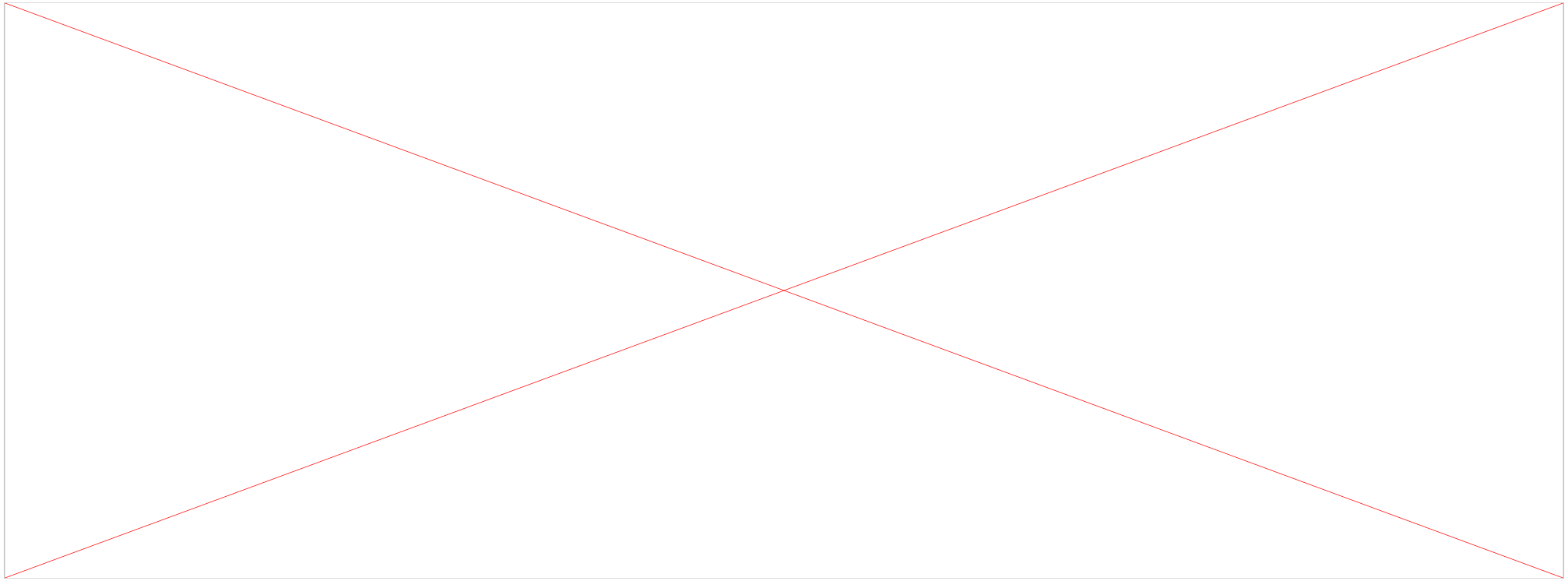


fig 3: RNN Unfolded

$$a_t = f(U * X_t + W * a_{t-1} + b)$$

Single RNN Cell

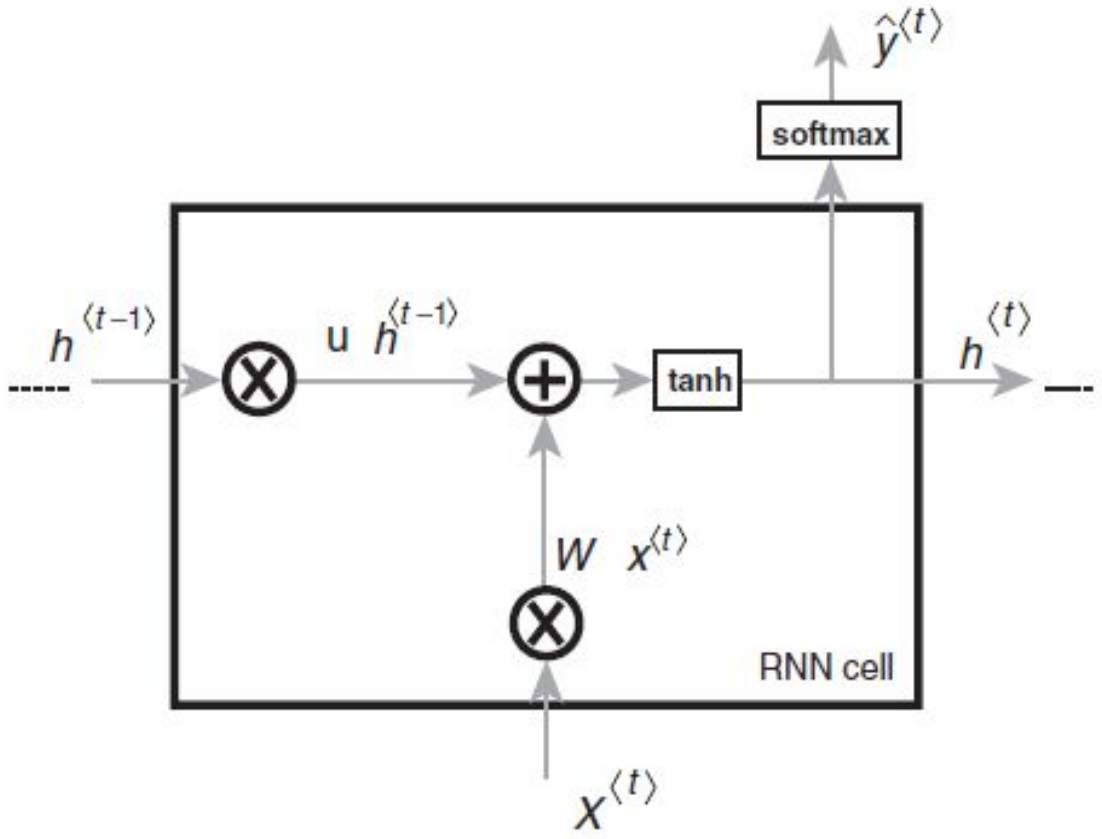


Fig. Single time step of an RNN cell

Structure of RNN.

CNN	RNN
It is suitable for spatial data such as images.	RNN is suitable for temporal data, also called sequential data.
CNN is considered to be more powerful than RNN.	RNN includes less feature compatibility when compared to CNN.
This network takes fixed size inputs and generates fixed size outputs.	RNN can handle arbitrary input/output lengths.
CNN is a type of feed-forward artificial neural network with variations of multilayer perceptrons designed to use minimal amounts of preprocessing.	RNN unlike feed forward neural networks - can use their internal memory to process arbitrary sequences of inputs.
CNNs use connectivity pattern between the neurons. This is inspired by the organization of the animal visual cortex, whose individual neurons are arranged in such a way that they respond to overlapping regions tiling the visual field.	Recurrent neural networks use time-series information - what a user spoke last will impact what he/she will speak next.
CNNs are ideal for images and video processing.	RNNs are ideal for text and speech analysis.

Structure of RNN.

Language Modeling Example

Task: Predict the next word or character given previous ones.

Example:

Input: "the cat sat on the"

Output: "mat“

the → [0.12, 0.55, -0.20, ...]
cat → [0.88, -0.11, 0.07, ...]
sat → [...]

"the" → RNN → h1
"cat" → RNN → h2
"sat" → RNN → h3
"on" → RNN → h4
"the" → RNN → h5

Steps:

1. Convert words → embeddings (dense vector that captures meaning)
2. Feed into the RNN one word at a time
3. Output probability distribution using softmax
4. Train via cross-entropy loss

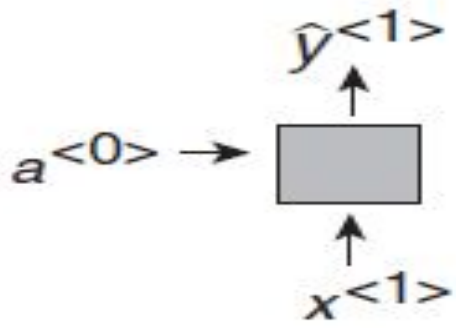
Word	Probability
mat	0.72
floor	0.10
bed	0.05
dog	0.01
...	...

RNN Topology

- One-to-One
- One-to-many
- Many-to-one
- Many-to-many for sequence input and sequence output
- Many-to-many for synced sequence input and output

One-to-One

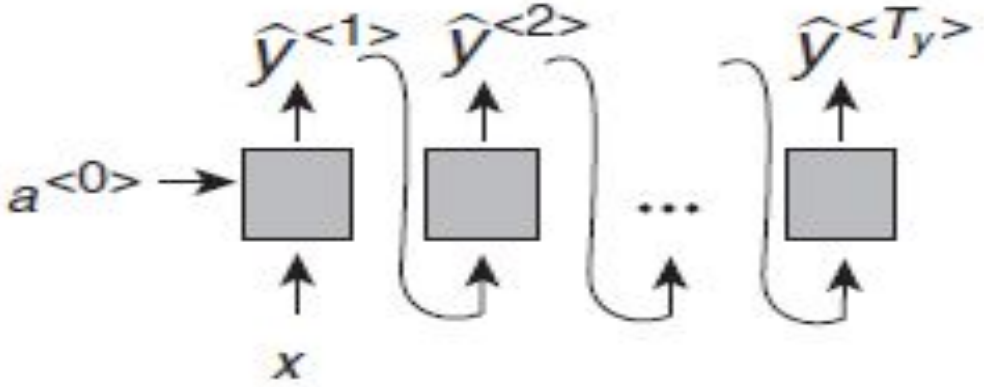
- **One-to-One** represents vanilla neural network (CNN) of processing without recurrent nets, from constant-sized input to constant-sized output (e.g., image classification).



One to one

One-to-Many

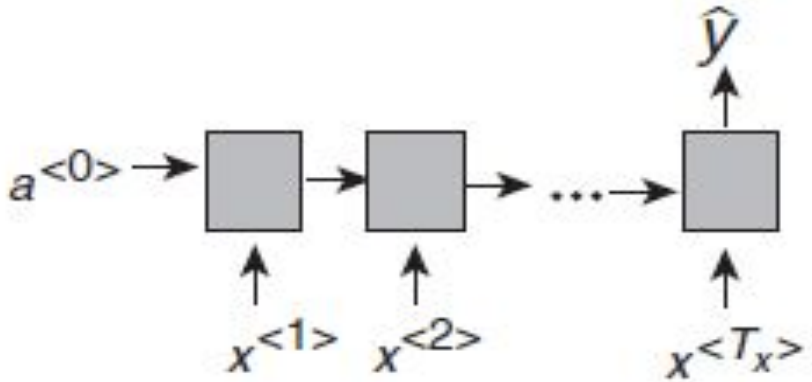
- A **one-to-many** neural network architecture refers to models where a **single, fixed-size input** is used to generate a **sequence of outputs**. (e.g., image captioning acquires an image as input and outputs a sentence of words).



One to many

Many-to-One

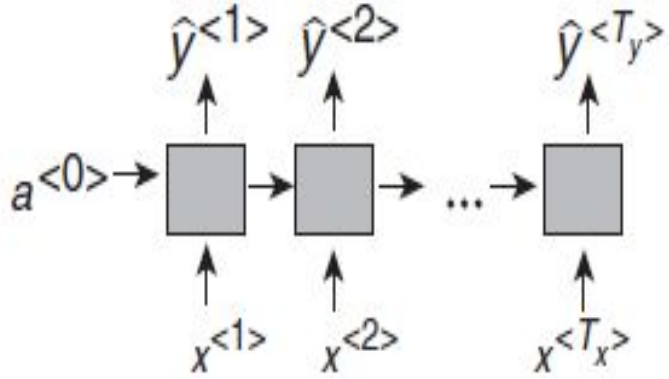
- A **many-to-one** neural network architecture processes a **sequence of inputs** and produces a **single output**.
- (e.g., sentiment analysis where a known sentence is classified as stating positive or negative sentiment).



Many to one

Many-to-Many

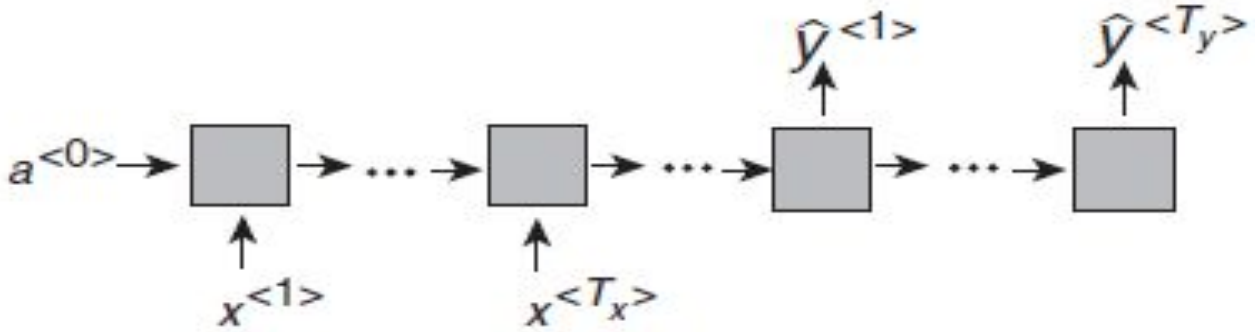
- A **many-to-many** neural network architecture handles **sequence inputs and sequence outputs**.
- A common example is **language translation**, where an RNN-based model (or modern Transformer) reads a sentence in English word-by-word and then generates a corresponding sentence in French word-by-word.
- Both the input and output are variable-length sequences.



Many to many

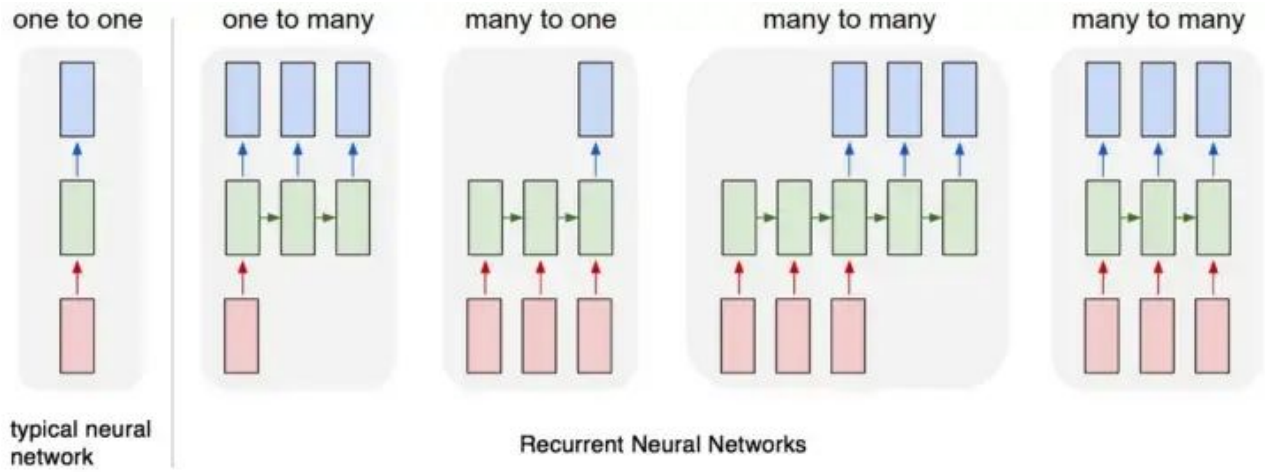
Many-to-Many (Contd…)

- **Many-to-many for synced sequence input and output, e.g., video classification,** where we want to label every frame.



Many to many

Summary of RNN topologies



- From left to right: (1) Vanilla mode of processing without RNN, from fixed-sized input to fixed-sized output (e.g. image classification).
- (2) Sequence output (e.g. image captioning takes an image and outputs a sentence of words).
- (3) Sequence input (e.g. sentiment analysis where a given sentence is classified as expressing positive or negative sentiment).
- (4) Sequence input and sequence output (e.g. Machine Translation: an RNN reads a sentence in English and then outputs a sentence in French).
- (5) Synced sequence input and output (e.g. video classification where we wish to label each frame of the video).
- Notice that in every case are no pre-specified constraints on the lengths sequences because the recurrent transformation (green) is fixed and can be applied as many times as we like.